

toychain

Блокчейн своїми руками на C++ — від нуля до власної монети. Видання для розуміння.

Перероблене видання. Побудоване на «Блокчейн своїми руками, покроковий гайд на C++», але з іншою механікою: спершу мова, потім блокчейн, і жодного готового коду, який можна переписати не думаючи.

Весь код цієї книги зібрано та запущено на macOS (Apple clang, OpenSSL 3, arm64). Розв'язки — в окремому файлі **toychain-solutions.pdf**. Відкривайте його лише після спроби.

Зміст

Як користуватися цією книгою	5
Пастка, у яку легко потрапити	5
Головне правило	5
Цикл на кожен розділ	5
Чому в книзі немає готового коду	6
Позначки	6
Про темп	7
Словник незнайомих слів	7
Що потрібно перед стартом	7
Крок 0. C++ рівно стільки, скільки потрібно	8
§0.1 Інструменти	8
§0.2 Що відбувається, коли ви «збираєте» програму	9
§0.3 Перша програма	9
§0.4 Типи та змінні	10
§0.5 const: «це не можна змінювати»	11
§0.6 std::string: рядок тексту	11
§0.7 Функції	11
§0.8 Копія, посилання, const-посилання	12
§0.9 Умови й цикли	13
§0.10 std::vector: список змінної довжини	13
§0.11 struct: власний тип	14
§0.12 Методи, class, public/private	14
§0.13 Конструктор і список ініціалізації	15
§0.14 const-метод	16
§0.15 Заголовки і .cpp	16
§0.16 Вказівники і nullptr	17
§0.17 Потоки	17
§0.18 Приведення типів	18
§0.19 static і throw	18
§0.20 Контрольна Кроку 0	19
Крок 1. Хеш: відбиток даних	20
Ідея	20
Нові слова	20
Контракт	21
Скелет	21
Перевірка	22
Зламай навмисне	23
Вправи	23

Крок 2. Блок	24
Ідея	24
Нові слова	24
Контракт	24
Скелет	25
Перевірка	26
Зламай навмисне	26
Вправи	27
Крок 3. Ланцюг і перевірка цілісності	28
Ідея	28
Нові слова	28
Контракт	29
Скелет	29
Перевірка	30
Зламай навмисне	31
Вправи	31
Крок 4. Proof of Work: майнінг	33
Ідея	33
Нові слова	33
Контракт	33
Скелет	34
Перевірка	36
Зламай навмисне	36
Крок 5. Транзакції та дерево Меркла	37
Ідея	37
Нові слова	38
Контракт	38
Скелет	39
Перевірка	41
Зламай навмисне	42
Крок 6. Гаманець і цифровий підпис	43
Ідея	43
Нові слова	43
Контракт	44
Скелет	45
Перевірка	48
Зламай навмисне	49
Вправи	49
Крок 7. Баланси і правила монети	50
Ідея	50
Нові слова	50
Контракт	51

Скелет	51
Перевірка	53
Зламай навмисне	54
Вправи	54
Крок 8. Збереження на диск	55
Ідея	55
Нові слова	55
Контракт	56
Скелет	56
Перевірка	59
Зламай навмисне	59
Вправи	60
Крок 9. Мережа: дві ноди	61
Ідея	61
Нові слова	61
Контракт	62
Скелет	62
Перевірка	64
Зламай навмисне	65
Що тут спрощено	66
Куди далі	67
Покращити touchain	67
Читати справжній код	67
Мова	67
Додаток А. Типові помилки та як їх читати	68
Компіляція	68
Лінкування	69
Під час виконання	69
Додаток Б. Шпаргалка	71
Файли проекту (фінальний стан)	71
П'ять перевірок is_valid()	71
Команди	71
Команди ноди	71
Додаток В. Словник понять C++	73
Додаток Г. Глосарій	74

Як користуватися цією книгою

Ця книга влаштована не так, як більшість технічних книг, і різниця не косметична. Перш ніж писати перший рядок коду, витратьте десять хвилин на цей розділ — інакше ви користуватиметесь нею так само, як користувалися попередньою, і результат буде той самий.

Пастка, у яку легко потрапити

Звичайна покрокова книга виглядає так: тут пояснення, під ним готовий код, далі результат запуску. Читач набирає код, бачить очікуваний вивід, ставить галочку і йде далі. Через дев'ять розділів у нього є працююча програма, яку він не може ні пояснити, ні змінити.

Причина проста і не має нічого спільного з лінню чи здібностями. Коли готовий код лежить перед очима, набирання його не вимагає розуміння — це робота рук. Мозок при цьому зайнятий розпізнаванням символів, а не побудовою моделі. Відчуття прогресу є, прогресу немає. Це називають **ілюзією засвоєння**: впізнати текст набагато легше, ніж відтворити його, і мозок плутає перше з другим.

Друга причина — конкретно про C++. Більшість книг «зроби блокчейн на C++» вчать блокчейну, а мову вважають відомою. Але C++ має шар, якого немає в жодній мові з першого досвіду початківця: заголовки й окрема компіляція, посилання, `const` у трьох різних ролях, ручне керування пам'яттю, C-бібліотеки з іншими правилами. Не знаючи цього шару, читач бачить не код, а заклинання.

Ця книга виправляє обидві причини. Спершу мова — рівно стільки, скільки треба, і не більше. Потім блокчейн — але без готових тіл функцій, які можна переписати не думаючи.

Головне правило

ПРАВИЛО, ВІД ЯКОГО ЗАЛЕЖИТЬ УСЕ

Не набирайте рядок, який не можете сказати вголос своїми словами.

Натрапили на незнайоме слово — це не перешкода на шляху до коду. Це і є ваша робота на зараз. Код почекає.

Правило звучить м'яко, але воно жорстке в застосуванні. `EVP_MD_CTX*` — ви можете сказати, що це? `const std::string&` — а це? Якщо ні, зупиніться. У книзі є розділ, де кожне з цих слів пояснено; знайдіть його і прочитайте. П'ять хвилин зараз економлять дев'ять розділів нерозуміння потім.

Цикл на кожен розділ

Порядок дій усередині кожного кроку. Порядок має значення — саме він відрізняє розуміння від переписування.

1. Прочитайте «Ідею» до кінця, перш ніж дивитись на код. Спочатку — навіщо, потім — як. Якщо ви не можете переказати ідею розділу однією фразою, код вам поки не потрібен.
2. Пройдіться таблицею «Нові слова». Кожен крок вводить кілька нових конструкцій мови. Вони зібрані в таблицю на початку розділу. Прочитайте всі, навіть ті, що здаються знайомими.
3. Подивіться на «Контракт» — заголовки. Заголовки в цій книзі даються повністю: це інтерфейс, домовленість про те, що і як називається. Їх набирайте як є.
4. Візьміть «Скелет» і напишіть тіла самі. Тіла функцій у книзі замінені на коментарі — покроковий опис того, що має статись. Ваше завдання — перетворити коментарі на код. Це і є основна робота.
5. Застрягли — не відкривайте розв’язок одразу. Спершу перечитайте «Нові слова» і відповідний параграф Кроку 0. Розв’язок — останній засіб, і після нього обов’язково закрийте його і напишіть код заново з пам’яті.
6. Заповніть «Передбач» до запуску. Запишіть — на папері, у файл, будь-де — що саме виведе програма. Тільки після цього запускайте. Не збіглося — ви щойно знайшли точне місце, де ваша модель хибна. Це найцінніший сигнал у всьому навчанні, і він безкоштовний.
7. Перевірте себе по «Еталону». Там, де вивід передбачуваний, книга дає точні значення. Там, де ні (час, випадкові ключі), — дає властивості, які мають виконуватись.
8. Дайте відповідь на «Самоперевірку» вголос. Не подумки. Вголос або письмово. Мовчазне «ну я ж розумію» — це та сама ілюзія засвоєння.
9. Зробіть «Зламай навмисне». Розуміти код = знати, що станеться, коли його зіпсувати. Кожен експеримент має передбачення: спершу напишіть своє, потім запустіть.

Чому в книзі немає готового коду

Тіла функцій навмисно замінені скелетами, а повний код винесено в окремий файл `toychain-solutions.pdf`. Це не спроба ускладнити життя. Це єдиний спосіб змусити працювати механізм, який справді вчить: **відтворення з пам’яті**.

Різниця величезна. Переписати відкритий текст — операція, яку виконують очі й пальці. Відтворити закритий — операція, яку виконує розуміння, і вона чесно повідомляє, коли розуміння немає. Що не згадали — саме там дірка. Ніякого самообману.

ЧОМУ САМЕ ТАК

Тому користуйтеся розв’язками так: спробували самі, застрягли, подивились **один потрібний фрагмент**, закрили файл, написали заново з пам’яті. Не «подивився і переписав» — «подивився, закрив, відтворив».

Позначки

Кольорові рамки означають різні типи роботи. Пропускати можна лише «Вправи».

ІДЕЯ

Навіщо це потрібно. Читати першим, до коду.

ТВОЄ ЗАВДАННЯ	Що саме написати. Основна робота розділу.
ПЕРЕДБАЧ	Записати очікуваний результат до запуску.
ЕТАЛОН	Точні значення або властивості для перевірки себе.
САМОПЕРЕВІРКА	Питання, на які треба відповісти вголос.
ЗЛАМАЙ	Навмисно зіпсувати код і пояснити наслідок.
ЧОМУ САМЕ ТАК	Пояснення рішення, яке могло бути іншим.
УВАГА	Пастка, безпека або незворотна дія.

Про темп

Дев'ять кроків цієї книги читаються за вечір. Засвоюються за місяць.

Реалістичний темп — **один крок за сеанс, іноді за два**. Крок 0 (мова) — два-три вечори. Далі по кроку. Крок 6 (гаманець і підпис) майже напевно займе два.

Це не повільно. Це нормальна швидкість, з якою людина будує нову модель у голові. Відчуття «я не розумію, що пишу» часто виникає саме тому, що читач намагався йти зі швидкістю читання — а вона фізично неможлива для засвоєння.

Пропускати кроки не можна: кожен наступний спирається на попередній, і половина розуміння Кроку 7 — це розуміння Кроку 5.

Словник незнайомих слів

Заведіть файл `UNKNOWN.md` у папці проєкту. Кожне слово, символ або конструкція, яку ви не можете пояснити, іде туди — не в голову, у файл. Формат простий:

`UNKNOWN.md`

- [] EVP_MD_CTX* — вказівник на щось з OpenSSL, не знаю на що
- [x] const& — посилання без копії, змінювати не можна (Крок 0, §7)

Цей файл — ваша навчальна програма, і головна його властивість у тому, що він **скорочується, а не росте**. Повний набір C++, потрібний для всієї книги, — приблизно шістнадцять понять. Вони всі в Кроці 0. Це не «половина мови», це скінченний список із видимим кінцем.

Що потрібно перед стартом

Комп'ютер з macOS (усе перевірено на Apple clang і OpenSSL 3), термінал, редактор і дві-три години на Крок 0. Усе встановлення описано в §0.1.

Якщо ви вже проходили якийсь інший гайд і у вас лишилися файли — почніть з чистої папки. Змішувати старий код, який ви не розумієте, з новим, який тільки будете, — надійний спосіб заплутатись.

Крок 0. C++ рівно стільки, скільки потрібно

Цей розділ — фундамент. Він не намагається навчити C++: мова величезна, і 90% її вам не знадобиться ані тут, ані найближчим часом. Він вчить рівно тим шістнадцятьом поняттям, з яких складений увесь код цієї книги, і зупиняється.

ІДЕЯ

Повний список того, що зустрінеться далі, — у таблиці нижче. Більше в книзі немає нічого. Немає шаблонів, немає `std::map`, немає лямбд, немає наслідування, `virtual`, розумних вказівників і `try/catch`. Коли пройдете цей розділ, ви знатимете все, що потрібно для дев'яти кроків.

слово	що означає
типи, змінні	<code>int</code> , <code>long</code> , <code>long long</code> , <code>bool</code> , <code>char</code> , <code>double</code> — §0.4
<code>const</code>	Три різні ролі однакового слова — §0.5, §0.8, §0.14
<code>std::string</code>	Рядок тексту — §0.6
функції	Оголошення, визначення, повернення — §0.7
<code>&</code> і <code>const&</code>	Посилання: робота без копіювання — §0.8
цикли	<code>for</code> , <code>while</code> , <code>for (... : ...)</code> — §0.9
<code>std::vector</code>	Список змінної довжини, <code>size_t</code> — §0.10
<code>struct</code>	Свій складений тип — §0.11
<code>class</code>	Те саме + приховані поля — §0.12
конструктор	Список ініціалізації — §0.13
метод <code>const</code>	Метод, що нічого не змінює — §0.14
<code>.h</code> / <code>.cpp</code>	Багатофайлова програма, лінування — §0.15
вказівники	<code>*</code> , <code>nullptr</code> , C-бібліотеки — §0.16
потоки	<code>cout</code> , <code>ostream</code> , <code>fstream</code> — §0.17
<code>(int)x</code>	Приведення типів — §0.18
<code>static</code> , <code>throw</code>	Видимість у файлі; виняток — §0.19

§0.1 Інструменти

Три речі: компілятор, OpenSSL і редактор.

Компілятор. На macOS він приходить з інструментами розробника Apple. Перевірка:

```
clang++ --version
```

Якщо команда не знайдена — виконайте `xcode-select --install` і погодьтесь на завантаження.

Homebrew і OpenSSL. Homebrew — менеджер пакетів для macOS. Якщо його немає, встановіть з brew.sh. Далі:

```
brew install openssl@3
brew --prefix openssl@3
```

Друга команда друкує шлях, куди встановилась бібліотека (зазвичай `/opt/homebrew/opt/openssl@3`). Запам'ятайте його — він знадобиться у скрипті збірки.

Редактор. Будь-який: VS Code, CLion, Vim, Zed. Для VS Code поставте розширення «C/C++» від Microsoft або «clang».

Папка проекту. Створіть порожню папку і в ній підпапку `src`:

```
mkdir -p ~/toychain/src && cd ~/toychain
```

§0.2 Що відбувається, коли ви «збираєте» програму

Це не деталь для допитливих. Половина помилок, які ви побачите, стають зрозумілими лише тоді, коли ви знаєте, на якому з трьох етапів вони виникли.

- 1. Препроцесор.** Тупий текстовий робот. Знаходить кожен рядок `#include <string>` і **фізично вставляє** на його місце весь вміст того файлу. `#pragma once` каже йому: «цей файл вставляй лише один раз». Після препроцесора кожен `.cpp` перетворився на один гігантський текст без жодного `#include`.
- 2. Компіляція.** Кожен такий текст окремо перетворюється на машинний код — файл `.o`. Компілятор перевіряє синтаксис і типи. Помилки на цьому етапі виглядають як `error: use of undeclared identifier` і завжди називають файл і рядок.
- 3. Лінкування.** Усі `.o` склеюються в один виконуваний файл. Лінкер шукає, де насправді лежить тіло кожної функції, яку хтось викликав. Помилки цього етапу виглядають зовсім інакше — `Undefined symbols` — і **не називають рядок**, бо проблема не в рядку, а у відсутньому файлі.

ЧОМУ САМЕ ТАК

Тому «функція оголошена, але не визначена» — це помилка лінкера, а не компілятора. Компілятор повірив вашому оголошенню в `.h` і пішов далі. Лінкер повірити не може: йому потрібне справжнє тіло. Це найчастіша плутанина в C++ у перші тижні.

§0.3 Перша програма

src/main.cpp

```
#include <iostream>

int main() {
    std::cout << "Привіт, toychain\n";
    return 0;
}
```

Розбір рядок за рядком:

слово	що означає
<code>#include <iostream></code>	Вставити сюди опис засобів вводу-виводу. Без цього <code>std::cout</code> не існує.
<code>int main()</code>	Точка входу. Виконання програми починається тут. <code>int</code> — тип того, що функція повертає.
<code>std::cout</code>	«Стандартний потік виводу» — консоль. <code>std::</code> — простір імен стандартної бібліотеки.
<code><<</code>	Оператор «відправити в потік». Читається зліва направо: у <code>cout</code> відправили рядок.
<code>\n</code>	Символ переходу на новий рядок.
<code>return 0;</code>	Код завершення. <code>0</code> = все добре. Саме для <code>main</code> його можна не писати — мова допише сама.

Збірка і запуск:

```
clang++ -std=c++17 -Wall -Wextra -o hello src/main.cpp
./hello
```

слово	що означає
<code>-std=c++17</code>	Версія стандарту мови. Уся книга — C++17.
<code>-Wall -Wextra</code>	Увімкнути попередження. Компілятор скажитиметься на підозріле — це добре.
<code>-o hello</code>	Назва виконуваного файлу на виході.

§0.4 Типи та змінні

У C++ кожна змінна має тип, і тип назавжди. Це головна відмінність від Python чи JavaScript.

приклад

```
int    index = 3;           // ціле, приблизно ±2 мільярди
long   timestamp = 1788999182; // довше ціле
long long amount = 50;      // ще довше: ±9 квінтільйонів
bool    ok = true;          // true або false
char    c = 'a';            // один байт
double  seconds = 0.45;     // дробове
```

УВАГА

Гроші ніколи не зберігають у `double`. Дробові числа в комп'ютері неточні: `0.1 + 0.2` не дорівнює `0.3`. У нашій монеті сума — це `long long`, ціла кількість монет. Bitcoin робить так само: усередині він рахує не біткоїни, а сатоші — цілі числа.

ТВОЄ ЗАВДАННЯ

Напишіть програму, яка створює по одній змінній кожного типу і друкує їх усі. Потім спробуйте записати в `int` значення `3000000000` і подивіться, що надрукується.

§0.5 const: «це не можна змінювати»

приклад

```
const int difficulty = 4;
difficulty = 5;    // помилка компіляції
```

const — обіцянка компілятору, що значення не зміниться. Компілятор цю обіцянку перевіряє і не дає її порушити.

Слово **const** зустрінеться у трьох різних ролях, і це головне джерело плутанини. Перша роль — тут. Дві інші — у §0.8 (параметр функції) і §0.14 (метод). Ролі не пов'язані між собою, крім спільної ідеї «щось тут незмінне».

§0.6 std::string: рядок тексту

приклад

```
#include <string>

std::string data = "Аліса -> Боб: 10";
std::string prev = "0000abc";

std::string content = data + prev;    // склеїти
size_t n = content.size();           // довжина
std::string head = content.substr(0, 4); // перші 4 символи
bool same = (data == prev);          // порівняти

std::string zeros(4, '0');            // "0000" — 4 копії символу
std::string s = std::to_string(42);   // число -> рядок "42"
```

слово	що означає
+	Склеювання рядків.
.size()	Кількість символів (точніше — байтів).
.substr(a, b)	Підрядок: b символів починаючи з позиції a . Позиції з нуля.
==	Порівняння вмісту, а не адрес. Працює так, як очікуєте.
string(n, c)	Рядок із n копій символу c . Знадобиться в майнінгу: <code>"0000"</code> .
std::to_string	Число в рядок. Зворотне — <code>std::stoi</code> , <code>std::stol</code> , <code>std::stoll</code> .

ТВОЄ ЗАВДАННЯ

Напишіть програму, яка бере рядок `"blockchain"`, друкує його довжину, перші 5 символів, і рядок з 10 зірочок.

§0.7 Функції

приклад

```
// оголошення: обіцянка, що така функція існує
int square(int x);
```

```
// визначення: тіло
int square(int x) {
    return x * x;
}
```

Перед круглими дужками — ім'я, після — параметри, перед іменем — тип того, що функція повертає. `void` означає «нічого не повертає».

Оголошення і визначення можна розділити — саме на цьому побудовані заголовкові файли (§0.15). Компілятору достатньо оголошення, щоб дозволити виклик; тіло знайде лінкер.

§0.8 Копія, посилання, const-посилання

Найважливіший параграф Кроку 0. Три способи передати щось у функцію.

приклад

```
void a(std::string s);           // копія
void b(std::string& s);          // посилання
void c(const std::string& s);    // const-посилання
```

Копія. Функція отримує власний дублікат. Зміни всередині не видно ззовні. Для рядка на мільйон символів це мільйон байтів роботи на кожен виклик.

Посилання (&). Функція працює з вашим оригіналом. Копіювання немає. Зміни всередині видно ззовні.

const&. Працює з оригіналом (швидко), але змінювати заборонено. Компілятор перевіряє.

ІДЕЯ

`const std::string& input` читається так: «дай мені доступ до твого рядка без копіювання; обіцяю не чіпати». Це стандартний спосіб передавати рядки і вектори в C++, і саме він зустрічається в цій книзі майже в кожній функції.

приклад

```
#include <iostream>
#include <string>

void by_copy(std::string s) { s += "!"; }
void by_ref(std::string& s) { s += "!"; }

int main() {
    std::string a = "hi";
    by_copy(a);
    std::cout << a << "\n";    // ?
    by_ref(a);
    std::cout << a << "\n";    // ?
}
```

ПЕРЕДБАЧ ДО ЗАПУСКУ

Що надрукують два `cout`? Запишіть обидва рядки, потім запустіть.

САМОПЕРЕВІРКА

Чому `const std::string&` швидший за `std::string`, хоча виглядає складніше? Скажіть відповідь вголос.

§0.9 Умови й цикли

приклад

```
if (hash.substr(0, 4) == "0000") {
    std::cout << "знайшли\n";
} else {
    std::cout << "далі\n";
}

for (int i = 0; i < 10; ++i) { /* ... */ }

while (nonce < 1000) { ++nonce; }
```

`++i` збільшує на одиницю. `!x` — «не x». `&&` — «і», `||` — «або».

Окремо — цикл по колекції, він зустрінеється десятки разів:

приклад

```
for (const std::string& s : names) {
    std::cout << s << "\n";
}
```

Читається: «для кожного `s` у `names`». `const&` тут з тієї ж причини, що й у §0.8: не копіювати кожен елемент і не змінювати його випадково.

§0.10 `std::vector`: список змінної довжини

приклад

```
#include <vector>

std::vector<std::string> level; // порожній список рядків
level.push_back("abc");        // додати в кінець
level.push_back("def");

size_t n = level.size();       // скільки елементів
std::string first = level[0];   // за індексом, з нуля
std::string last = level.back(); // останній
level.clear();                  // спорожнити
bool empty = level.empty();
```

слово	що означає
<code>vector<T></code>	Список елементів типу <code>T</code> . <code>T</code> пишеться в кутових дужках.
<code>.push_back(x)</code>	Додати <code>x</code> у кінець; вектор сам виросте.
<code>.size()</code>	Кількість елементів. Тип результату — <code>size_t</code> .

слово	що означає
<code>size_t</code>	Ціле без знаку для розмірів. Ніколи не буває від'ємним — тому цикли по векторах пишуть <code>size_t i = 0</code> .
<code>[i]</code>	Доступ за індексом. Перевірки меж немає: вихід за межі — це не помилка, а невизначена поведінка.
<code>.back()</code>	Останній елемент.

УВАГА

`level[10]` у векторі з трьох елементів не викине помилку. Програма прочитає чужу пам'ять і поведеться непередбачувано — може впасти, а може мовчки видати сміття. Це називають **undefined behavior**, і в C++ воно скрізь, де немає перевірки.

ТВОЄ ЗАВДАННЯ

Створіть `std::vector<int>`, покладіть у нього числа від 1 до 10, потім пройдіться циклом і надрукуйте лише парні. Далі надрукуйте `.size()` і `.back()`.

§0.11 struct: власний тип

Коли кілька значень описують одну річ, їх об'єднують.

приклад

```
struct Transaction {
    std::string from;
    std::string to;
    long long amount;
};

Transaction tx;
tx.from = "alice";
tx.amount = 10;

// або одразу, значеннями по порядку полів:
Transaction tx2{"alice", "bob", 10};
```

Крапка з комою після закривальної фігурної дужки `struct` — обов'язкова. Її забувають усі, і компілятор скаржиться `expected ';' after struct`.

§0.12 Методи, class, public/private

Структура може містити не лише поля, а й функції, які працюють з цими полями. Такі функції називають **методами**.

приклад

```
struct Transaction {
    std::string from;
    std::string to;
    long long amount;
```

```

    std::string to_string(); // метод
};

// визначення методу — за межами структури:
std::string Transaction::to_string() {
    return from + "->" + to + ":" + std::to_string(amount);
}

```

Transaction:: перед іменем каже: «це не вільна функція, це метод цієї структури». У середині методу поля видно напряду, без **tx.** — метод завжди працює з конкретним об'єктом.

class проти **struct**. Різниця рівно одна: у **struct** усе публічне за замовчуванням, у **class** — приватне. Приватне видно лише зсередини самого класу.

приклад

```

class Wallet {
public:
    std::string address(); // видно всім

private:
    EVP_PKEY* key_; // видно лише методам Wallet
};

```

Підкреслення в кінці **key_** — просто угода про іменування, щоб було видно: це поле, а не локальна змінна. Мова про це нічого не знає.

§0.13 Конструктор і список ініціалізації

Конструктор — метод, який виконується при створенні об'єкта. Називається так само, як тип, і нічого не повертає.

приклад

```

struct Block {
    int index;
    std::string data;
    long nonce;

    Block(int index, const std::string& data);
};

Block::Block(int index, const std::string& data)
: index(index), // <- список ініціалізації
  data(data),
  nonce(0) {
    // тіло конструктора
}

```

Двокрапка після дужок відкриває **список ініціалізації членів**. Він виконується до тіла конструктора.

ЧОМУ САМЕ ТАК

Різниця між списком і тілом не косметична. У списку поля **створюються** одразу з потрібним значенням. У тілі вони спершу створюються порожніми, а потім їм **присвоюють**. Для **std::string** це зайве виділення пам'яті на кожен об'єкт.

`index(index)` виглядає дивно, але однозначно: **зовнішнє** — завжди поле, **внутрішнє в дужках** — параметр. У цій позиції компілятор тлумачить це лише так.

Коли значення поля залежить від інших полів, його рахують у тілі — там усі поля вже готові.

§0.14 const-метод

приклад

```
struct Block {
    std::string hash;
    std::string calculate_hash() const;    // <- const ПІСЛЯ дужок
};
```

`const` після дужок — третя роль цього слова: «цей метод не змінює об'єкт, лише читає». Спроба присвоїти полю всередині такого методу — помилка компіляції:

вивід

```
error: cannot assign to non-static data member within const member function
```

Правило просте: якщо метод нічого не змінює — позначайте `const`. Це і документація, і перевірка, і умова для того, щоб метод можна було викликати на `const&`-об'єкті.

§0.15 Заголовки і .cpp

Програма з багатьох файлів — не примха, а необхідність: інакше зміна одного рядка вимагала б перекомпіляції всього.

Правило поділу: у `.h` — **що** є (оголошення), у `.cpp` — **як** воно працює (тіла).

src/block.h

```
#pragma once
#include <string>

struct Block {
    int index;
    std::string calculate_hash() const;
};
```

src/block.cpp

```
#include "block.h"

std::string Block::calculate_hash() const {
    return "поки що заглушка";
}
```

слово	що означає
<code>#pragma once</code>	Вставити цей файл лише один раз. Без нього подвійне включення = помилка «перевизначення».
<code>#include <...></code>	Кутові дужки — системний або бібліотечний заголовок.
<code>#include "..."</code>	Лапки — ваш власний файл, поруч.

УВАГА

Кожен `.cpp` треба передати компілятору. Забули `block.cpp` у команді збірки — отримаєте не помилку компіляції, а помилку **лінкера**: `Undefined symbols`. Скрипт `build.sh` у цій книзі бере `src/*.cpp` — усі файли одразу, тому проблема не виникає.

§0.16 Вказівники і `nullptr`

Вказівник — змінна, яка зберігає адресу іншого об'єкта.

приклад

```
int x = 5;
int* p = &x;    // p зберігає адресу x
std::cout << *p; // 5 — розіменування: "значення за адресою"
p = nullptr;    // "нікуди не вказує"
```

Зірочка в оголошенні (`int* p`) означає «вказівник». Зірочка перед іменем (`*p`) — «візьми значення за цією адресою». `&x` — «візьми адресу x».

ІДЕЯ

У цій книзі вказівники вам майже не потрібні **самі по собі**. Вони потрібні тому, що OpenSSL — бібліотека на C, а C не має ні конструкторів, ні деструкторів, ні посилань. Там усе передається вказівниками, а пам'ять звільняється руками:

```
EVP_MD_CTX* ctx = EVP_MD_CTX_new();    // виділити
// ... робота ...
EVP_MD_CTX_free(ctx);                  // звільнити — обов'язково
```

Забули `free` — витік пам'яті. У майнінгу, де хеш рахується мільйони разів, такий витік з'їсть пам'ять за секунди.

Ще одна роль вказівника в C-бібліотеках — **вихідний параметр**. Функція не може повернути два значення, тому друге вона записує за переданою адресою:

приклад

```
unsigned int length = 0;
EVP_DigestFinal_ex(ctx, digest, &length);
// після виклику length містить справжню довжину
```

§0.17 Потоки

`std::cout` — окремий випадок загальнішої ідеї: об'єкт, у який пишуть через `<<` або з якого читають через `>>`.

приклад

```
#include <iostream>    // cout, cin
#include <sstream>      // ostringstream, istringstream
#include <fstream>      // ifstream, ofstream

// зібрати рядок частинами
std::ostringstream out;
```

```

out << "hash=" << 42 << ";";
std::string s = out.str();      // "hash=42;"

// розібрати рядок на частини
std::istringstream in("send alice 10");
std::string cmd, who;
long long amount;
in >> cmd >> who >> amount;    // "send", "alice", 10

// файли
std::ofstream file("chain.txt");
file << "перший рядок\n";

std::ifstream input("chain.txt");
std::string line;
while (std::getline(input, line)) { /* по рядку */ }

```

Окремо — **маніпулятори**, які змінюють формат виводу:

приклад

```

#include <iomanip>

out << std::hex;           // далі числа в 16-й системі
out << std::setw(2);       // ширина поля 2
out << std::setfill('0');   // добувати нулями

```

УВАГА

`std::hex` і `std::setfill` — «липкі»: діють до скасування. `std::setw` — **ні**: він скидається після кожного виводу. Тому в коді, який перетворює байти в hex, весь ланцюжок стоїть усередині циклу, а не перед ним.

§0.18 Приведення типів

приклад

```

unsigned char b = 15;
std::cout << b;           // надрукує символ, а не число
std::cout << (int)b;      // 15
std::cout << static_cast<int>(b); // те саме, довша й безпечніша форма

```

`char` для потоку — це **літера**, не число. Тому байт `0x0f` без приведення надрукується як невидимий службовий символ. Це рівно та пастка, через яку в Кроці 1 хеш перетворився б на сміття.

`(int)x` — коротка форма, `static_cast<int>(x)` — сучасна. Обидві в книзі зустрінуться.

§0.19 static і throw

static на рівні файлу означає «ця функція або змінна видима лише всередині цього `.cpp`». Так ховають допоміжні функції, щоб вони не конфліктували з однойменними з інших файлів.

приклад

```

static std::string to_hex(const std::vector<unsigned char>& bytes) { /* ... */ }

```

static усередині класу — інше: поле, спільне для всіх об'єктів класу.

приклад

```
class Blockchain {  
    static const long long MINING_REWARD = 50; // одне на весь клас  
};
```

throw кидає виняток — аварійно перериває виконання. У цій книзі він трапляється двічі, там, де продовжувати безглуздо (гаманець без ключа). Ловити винятки ми не будемо.

§0.20 Контрольна Кроку 0

Не переходьте до Кроку 1, поки не зробите це. Одна програма, яка використовує все з розділу.

ТВОЄ ЗАВДАННЯ

Напишіть проєкт із трьох файлів: `src/wallet_demo.h`, `src/wallet_demo.cpp`, `src/main.cpp`.

1. У заголовку оголошіть `struct Person` з полями `std::string name` і `long long coins`, конструктор `Person(const std::string& name, long long coins)` і метод `std::string describe() const`.
2. У `.cpp` напишіть конструктор через список ініціалізації і метод `describe()`, який повертає рядок виду `"alice: 50 монет"`.
3. У `main` створіть `std::vector<Person>`, покладіть трьох людей, пройдіться циклом `for (const Person& p : people)` і надрукуйте опис кожного.
4. Додайте вільну функцію `long long total(const std::vector<Person>& people)`, яка повертає суму монет усіх.
5. Зберіть усе однією командою `clang++ -std=c++17 -Wall -Wextra -o demo src/*.cpp`.

САМОПЕРЕВІРКА

Відповідайте вголос. Якщо на якесь питання відповіді немає — поверніться до відповідного параграфа.

1. Чому `const std::string&` краще за `std::string` у параметрі функції?
2. Чим `const` після дужок методу відрізняється від `const` перед типом параметра?
3. Що станеться, якщо забути `#pragma once` у заголовку, який включають два `.cpp`?
4. На якому етапі — компіляції чи лінування — виникає `Undefined symbols`, і чому?
5. Навіщо `std::to_string`, якщо є `std::cout << 42`?
6. Чому в списку ініціалізації `index(index)` не є нескінченним циклом?
7. Чому суму грошей зберігають у `long long`, а не в `double`?

ІДЕЯ

Зробили контрольну і відповіли на всі сім питань — Крок 0 закінчено. Далі мова вже не буде перешкодою; далі буде блокчейн.

Крок 1. Хеш: відбиток даних

Мета розділу: функція `sha256(текст)`, що повертає 64 хеш-символи. Це фундамент — усе інше в блокчейні є хешами, з'єднаними певним чином.

Ідея

Хеш-функція бере будь-який текст і повертає рядок фіксованої довжини. У SHA-256 це завжди 32 байти, які ми показуємо як 64 символи `0–9`, `a–f`.

Три властивості, заради яких вона існує:

- 1. Детермінованість.** Той самий вхід завжди дає той самий хеш. На будь-якому комп'ютері, у будь-якій мові, через сто років.
- 2. Лавинний ефект.** Зміна одного біта на вході змінює приблизно половину бітів на виході. Схожі тексти не дають схожих хешів — хеші виглядають абсолютно різними.
- 3. Незворотність.** З хеша не можна відновити текст. Єдиний спосіб — перебирати варіанти, а їх астрономічно багато.

Разом це дає «відбиток»: короткий рядок, який однозначно свідчить про вміст, але нічого про нього не розповідає. Саме на цьому побудований весь блокчейн.

Сам алгоритм SHA-256 писати не треба — він є в OpenSSL. Наша робота: акуратно його викликати і перетворити 32 сирих байти на читабельний рядок.

ЧОМУ САМЕ ТАК

Навіщо взагалі переводити в хеш? Байти `0x2c 0xf2 0x4d ...` у консолі виглядають як сміття (більшість з них — недруковані символи), їх не можна порівняти очима, скопіювати чи покласти в текстовий файл. Шістнадцяткове представлення робить хеш звичайним рядком.

Нові слова

слово	що означає
<code>EVP_MD_CTX</code>	«Контекст» OpenSSL — об'єкт, у якому накопичується стан обчислення хеша.
<code>EVP_MD_CTX_new()</code>	Створює контекст у купі, повертає вказівник. Пара до нього — <code>_free</code> .
<code>EVP_sha256()</code>	Каже, який саме алгоритм використовувати. Замініть на <code>EVP_sha512()</code> — і решта коду не зміниться.
<code>Init / Update / Final</code>	Стандартний ритм OpenSSL: налаштувати, згодувати дані (можна частинами), забрати результат.
<code>EVP_MAX_MD_SIZE</code>	Константа = 64. Розмір буфера «з запасом»: SHA-256 заповнить лише 32 байти.

слово	що означає
<code>unsigned char[]</code>	Масив байтів фіксованої довжини на стеку.
<code>.data(), .size()</code>	Вказівник на сирі байти рядка і їх кількість. С-функції не знають, що таке <code>std::string</code> .
<code>std::ostringstream</code>	«Рядок, у який можна писати через <code><<</code> » — див. §0.17.
<code>std::hex / setw / setfill</code>	Маніпулятори формату. <code>setw</code> — не липкий! див. §0.17.
<code>(int)digest[i]</code>	Приведення: без нього байт друкується як літера, а не як число — §0.18.

Контракт

Заголовок набирайте як є — це домовленість, а не робота.

`src/sha256.h`

```
#pragma once
#include <string>

// Повертає SHA-256 від рядка у вигляді 64 hex-символів.
std::string sha256(const std::string& input);
```

Скрипт збірки — теж інфраструктура, набирайте як є:

`build.sh`

```
#!/bin/sh
set -e
OPENSSL="$(brew --prefix openssl@3)"
clang++ -std=c++17 -Wall -Wextra \
  -I"$OPENSSL/include" -L"$OPENSSL/lib" -lcrypto \
  -o toychain src/*.cpp
echo "OK: ./toychain"
```

`chmod +x build.sh`

слово	що означає
<code>-I</code>	Де шукати заголовки (<code>openssl/evp.h</code>).
<code>-L</code>	Де шукати саму бібліотеку.
<code>-lcrypto</code>	Приєднати бібліотеку <code>libcrypto</code> — це та частина OpenSSL, де живе SHA-256.
<code>src/*.cpp</code>	Усі <code>.cpp</code> одразу, щоб не забути жоден — див. попередження в §0.15.

Скелет

`src/sha256.cpp` · тіла написати самому

```
#include "sha256.h"

#include <openssl/evp.h>
#include <iomanip>
#include <sstream>
```

```
std::string sha256(const std::string& input) {
    // 1. Оголосити масив `digest` типу unsigned char на EVP_MAX_MD_SIZE байтів
    //    і змінну `length` типу unsigned int, ініціалізовану нулем.

    // 2. Створити контекст: EVP_MD_CTX_new().

    // 3. EVP_DigestInit_ex(контекст, EVP_sha256(), nullptr)

    // 4. EVP_DigestUpdate(контекст, вказівник на дані, скільки байтів)
    //    Дані беруться з input: .data() і .size().

    // 5. EVP_DigestFinal_ex(контекст, digest, &length)
    //    Після цього в digest лежать байти, у length – скільки їх.

    // 6. Звільнити контекст: EVP_MD_CTX_free(...). Не пропускайте цей крок.

    // 7. Заведіть std::ostringstream і в циклі від 0 до length
    //    відправте в нього кожен байт як два hex-символи:
    //    std::hex, std::setw(2), std::setfill('0') і приведення до int.

    // 8. Поверніть .str() від потоку.
}
```

src/main.cpp

```
#include <iostream>
#include "sha256.h"

int main() {
    std::cout << "hello -> " << sha256("hello") << "\n";
    std::cout << "hello! -> " << sha256("hello!") << "\n";
    std::cout << "(пусто) -> " << sha256("") << "\n";
    return 0;
}
```

ТВОЄ ЗАВДАННЯ

1. Створіть `build.sh`, `src/sha256.h`, `src/main.cpp` як показано вище.
2. Напишіть тіло `sha256` за скелетом. Не підглядайте в розв'язки, поки не спробували.
3. Зберіть і запустіть: `./build.sh && ./toychain`.

Перевірка

ПЕРЕДБАЧ ДО ЗАПУСКУ

Перед запуском дайте відповідь: чи будуть хеші `"hello"` і `"hello!"` схожими між собою? Скільки перших символів, на вашу думку, збігатиметься?

ЕТАЛОН ДЛЯ САМОПЕРЕВІРКИ

Ці три значення — не випадкові числа, а всесвітньо відомі константи. Будь-яка правильна реалізація SHA-256 у будь-якій мові дасть саме їх:

```
hello -> 2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e73043362938b9824
hello! -> ce06092fb948d9ffac7d1a376e404b26b7575bcc11ee05a4615fef4fec3a308b
(пусто) -> e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855
```

Незалежна перевірка прямо в терміналі:

```
echo -n hello | shasum -a 256
```

Якщо ваші значення збігаються — функція правильна, і можна йти далі. Якщо ні — майже напевно проблема в кроці 7 скелета; переходьте до експериментів нижче, вони покажуть, у чому.

САМОПЕРЕВІРКА

1. Навіщо `EVP_MD_CTX_free`? Що станеться, якщо його прибрати?
2. Чому `EVP_DigestUpdate` приймає `input.data()` і `input.size()`, а не сам `input`?
3. Чому масив `digest` має розмір 64, якщо SHA-256 дає 32 байти?
4. Що таке `length` і хто його заповнює?

Зламай навмисне

ЗЛАМАЙ НАВМИСНЕ

Експеримент 1. Приберіть `(int)` перед `digest[i]`. Запишіть передбачення, запустіть, поясніть побачене. (Підказка: `char` для потоку — літера, не число, §0.18.)

Експеримент 2. Приберіть `std::setw(2)` і `std::setfill('0')`. Скільки символів тепер у хеші `"hello"`? Підказка: серед його 32 байтів є байт зі значенням менше 16, який займе один символ замість двох. Порахуйте, скільки таких.

Експеримент 3. Винесіть `std::hex << std::setw(2) << std::setfill('0')` за межі циклу, залишивши в циклі лише `(int)digest[i]`. Хеш зіпсується — чому? Який із трьох маніпуляторів «нелипкий»?

Експеримент 4. Замініть `EVP_sha256()` на `EVP_sha512()`. Що зміниться у виводі і чому решта коду продовжила працювати без правок?

Вправи

1. Додайте виклик `sha256` для свого імені. Змініть одну літеру і порівняйте — скільки символів збіглося? Це і є лавинний ефект.
2. Напишіть цикл, що хешує рядки `"0"`, `"1"`, ..., `"99999"` і рахує, скільки з них починаються на `"00"`. Порівняйте з очікуванням $1/256$ — це попередній перегляд майнінгу.

Крок 2. Блок

Мета розділу: структура `Block` з полями і методом `calculate_hash()`, і перший досвід «підробки», яку хеш ловить.

Ідея

Блок — це просто структура з полями. Головне рішення в ній одне: **хеш блоку рахується від усіх полів разом**.

Звідси два різні поняття, які легко переплутати:

- поле `hash` — хеш, збережений **на момент створення**;
- метод `calculate_hash()` — хеш, порахований **заново з поточних полів**.

Поки блок ніхто не чіпав, вони збігаються. Змінили хоч біт у будь-якому полі — розійшлися. Це і є весь механізм цілісності одного блоку.

Поле `nonce` поки що завжди нуль. Воно запрацює в Кроці 4. Додаємо його вже зараз, щоб потім не переписувати формулу хеша: інакше всі раніше пораховані хеші стали б іншими.

ЧОМУ САМЕ ТАК

Чому поле `hash` не входить у власний розрахунок? Бо це замкнене коло: хеш залежав би від самого себе, і порахувати його було б неможливо. Хеш — це відбиток **усього іншого**.

Нові слова

слово	що означає
<code>struct</code>	Свій складений тип — §0.11.
<code>Block::Block(...)</code>	Конструктор, визначений за межами структури — §0.13.
<code>: index(index), ...</code>	Список ініціалізації членів — §0.13.
<code>std::time(nullptr)</code>	Поточний час у секундах з 1 січня 1970. <code>nullptr</code> = «не копіюй результат ще й у мою змінну».
<code>const</code> після дужок	Метод нічого не змінює — §0.14.
<code>std::to_string</code>	Число в рядок: <code>sha256</code> приймає рядок, а <code>index</code> — число.

Контракт

src/block.h

```
#pragma once
#include <string>

struct Block {
```

```

int index;           // номер блоку в ланцюгу (0 = перший)
long timestamp;      // час створення, секунди з 1970 року
std::string data;     // корисне навантаження (поки що просто текст)
std::string prev_hash; // хеш попереднього блоку
long nonce;          // "лічильник" для майнінгу (поки не використовуємо)
std::string hash;     // хеш цього блоку

Block(int index, const std::string& data, const std::string& prev_hash);

// Рахує хеш з усіх полів (крім самого hash).
std::string calculate_hash() const;
};

```

Скелет

src/block.cpp · тіла написати самому

```

#include "block.h"
#include "sha256.h"

#include <ctime>

Block::Block(int index, const std::string& data, const std::string& prev_hash)
    // Список ініціалізації: index, timestamp, data, prev_hash, nonce.
    // - index, data, prev_hash беруться з параметрів;
    // - timestamp = std::time(nullptr);
    // - nonce = 0.
{
    // У тілі: порахувати hash через calculate_hash().
    // Подумайте, чому саме в тілі, а не в списку ініціалізації.
}

std::string Block::calculate_hash() const {
    // 1. Склейте в один рядок `content`: index, timestamp, data, prev_hash, nonce.
    // Числа доведеться перетворити через std::to_string.
    // 2. Поверніть sha256(content).
}

```

src/main.cpp

```

#include <iostream>
#include "block.h"

int main() {
    Block block(1, "Аліса платить Бобу 10 монет", "0000abc");

    std::cout << "index:      " << block.index << "\n";
    std::cout << "timestamp: " << block.timestamp << "\n";
    std::cout << "data:      " << block.data << "\n";
    std::cout << "prev_hash: " << block.prev_hash << "\n";
    std::cout << "hash:      " << block.hash << "\n\n";

    // Спробуємо "підправити" дані вже після створення блоку.
    block.data = "Аліса платить Бобу 1000 монет";
    std::cout << "збережений hash:      " << block.hash << "\n";
    std::cout << "перерахований hash: " << block.calculate_hash() << "\n";
    std::cout << (block.hash == block.calculate_hash()
        ? "блок цілий" : "блок ПІДРОБЛЕНО") << "\n";
}

```

```
    return 0;
}
```

ТВОЄ ЗАВДАННЯ

1. Наберіть `block.h` як є.
2. Напишіть `block.cpp` за скелетом — і список ініціалізації, і обидва тіла.
3. Замініть `main.cpp` на показаний вище, зберіть, запустіть.

Перевірка

ПЕРЕДБАЧ ДО ЗАПУСКУ

Два питання до запуску:

1. Чи буде `hash` починатися з `0000`, якщо `prev_hash` дорівнює `"0000abc"`?
2. Після зміни `block.data` — які два рядки виведе програма, і що надрукується в третьому?

ЕТАЛОН ДЛЯ САМОПЕРЕВІРКИ

Точних значень тут немає: `timestamp` у вас інший, тому й хеші інші. Перевіряйте властивості:

- `hash` — рівно 64 символи;
- до зміни `data` збережений і перерахований хеш збігаються;
- після зміни — розходяться, і збігу немає **в жодному** символі з початку;
- останній рядок: **блок ПІДРОБЛЕНО**.

САМОПЕРЕВІРКА

1. Чому `hash = calculate_hash()` стоїть у тілі конструктора, а не в списку ініціалізації?
2. Що означає `const` після дужок `calculate_hash()`?
3. Чому в рядок `content` не входить саме поле `hash`?
4. Чи важливий порядок склеювання полів у `content`? Чому?

Зламай навмисне

ЗЛАМАЙ НАВМИСНЕ

Експеримент 1. Поміняйте місцями `data` і `prev_hash` у `content`. Перезберіть, запустіть двічі. Чи змінився хеш того самого блоку? Що це говорить про сумісність двох версій програми?

Експеримент 2. Приберіть `timestamp` зі склеювання. Створіть два блоки з однаковими `index`, `data`, `prev_hash`. Чи однакові в них хеші? Чому це погано?

Експеримент 3. Створіть два блоки з однаковими даними в одну секунду, але з різними `index`. Однакові хеші?

Експеримент 4 — справжня діра в цьому коді. Поля склеюються **без роздільника**. Блок з `index = 1`, `timestamp = 12` дає рядок, що починається на `"112"`. Блок з `index = 11`, `timestamp = 2` дає `"112"` теж. Два різні блоки — однаковий вміст для хеша. Вставте `"|"` між полями і поясніть, що саме це виправляє.

У справжньому блокчейні цієї проблеми немає, бо поля мають фіксовану довжину в байтах і склеювати їх однозначно. Наша іграшка склеює текст — і тому потребує роздільника.

Вправи

1. Додайте метод `void print() const`, який друкує всі поля блоку в один рядок.
2. Зробіть `data` полем типу `long long` замість рядка і подивіться, що доведеться змінити в `calculate_hash()`.

Крок 3. Ланцюг і перевірка цілісності

Мета розділу: клас `Blockchain`, який тримає вектор блоків, додає нові з правильним `prev_hash` і вміє перевірити себе.

Ідея

Одинокий блок від підробки не захищає: зловмисник змінив `data`, викликав `calculate_hash()`, перезаписав `hash` — і сліди заметено. Захищає **ланцюг**.

Ланцюг — це `std::vector<Block>` плюс два правила:

1. перший блок (**генезис**) створюється в конструкторі з `prev_hash = "0"`;
2. кожен наступний отримує `prev_hash` = хеш останнього блоку.

Перевірка `is_valid()` проходить по всіх блоках, починаючи з другого, і кожному ставить два запитання:

- «чи твій `hash` збігається з твоїм вмістом?»
- «чи твій `prev_hash` збігається з хешем попередника?»

Одне «ні» — ланцюг підроблено.

ІДЕЯ

Друге запитання і є суттю блокчейну. Зловмисник, що змінив блок 1 і чесно перерахував його хеш, натикається на блок 2: той **пам'ятає старий** хеш блоку 1. Щоб довести підробку до кінця, треба перерахувати блок 2, потім 3, і так до останнього.

Нові слова

слово	що означає
<code>class</code>	Те саме, що <code>struct</code> , але поля за замовчуванням приватні — §0.12.
<code>public:</code>	Далі йде те, що видно ззовні.
<code>std::vector<Block></code>	Список блоків — §0.10.
<code>.push_back(x)</code>	Додати блок у кінець.
<code>.back()</code>	Останній елемент вектора.
<code>const Block&</code>	Повернути посилання на блок, не копіюючи його — §0.8.
<code>size_t i</code>	Тип для індексів і розмірів — §0.10.
<code>blocks.size()</code>	Кількість блоків. Вона ж — індекс наступного: якщо блоків 3 (індекси 0,1,2), новий отримає 3.
<code>for (const Block& b : blocks)</code>	Цикл по колекції — §0.9.

Контракт

src/blockchain.h

```
#pragma once
#include <string>
#include <vector>
#include "block.h"

class Blockchain {
public:
    Blockchain(); // створює генезис-блок

    void add_block(const std::string& data);
    bool is_valid() const;
    const Block& last_block() const;
    void print() const;

    std::vector<Block> blocks; // публічне, щоб у прикладах легко "ламати" ланцюг
};
```

ЧОМУ САМЕ ТАК

Поле `blocks` навмисно публічне. У справжній бібліотеці воно було б приватним, але нам треба вміти ламати ланцюг зсередини, щоб побачити, як він себе захищає.

Скелет

src/blockchain.cpp · тіла написати самому

```
#include "blockchain.h"
#include <iostream>

Blockchain::Blockchain() {
    // Створіть генезис-блок: index 0, дані "Genesis", prev_hash "0",
    // і покладіть його у blocks.
}

const Block& Blockchain::last_block() const {
    // Поверніть останній блок вектора.
}

void Blockchain::add_block(const std::string& data) {
    // 1. Створіть блок: номер = blocks.size(), дані = data,
    //    prev_hash = хеш останнього блоку.
    // 2. Покладіть його у blocks.
}

bool Blockchain::is_valid() const {
    // Цикл від i = 1 (не від 0! у генезиса немає попередника) до blocks.size().
    // Для кожного блоку візьміть посилання на поточний і на попередній, і перевірте:
    //
    //    1. current.hash == current.calculate_hash()
    //    інакше: надрукувати "Блок i: hash не збігається з вмістом" і повернути
    false
    //
    //    2. current.prev_hash == previous.hash
    //    інакше: надрукувати "Блок i: prev_hash не збігається з попереднім блоком"
    //    і повернути false
}
```

```
//
// Дійшли до кінця — повернути true.
}

void Blockchain::print() const {
    // Цикл по всіх блоках. Для кожного надрукувати номер, data
    // і перші 16 символів hash та prev_hash (підказка: .substr(0, 16)).
}
```

src/main.cpp

```
#include <iostream>
#include "blockchain.h"

int main() {
    Blockchain chain;
    chain.add_block("Аліса -> Боб: 10");
    chain.add_block("Боб -> Карл: 5");
    chain.add_block("Карл -> Аліса: 2");

    chain.print();
    std::cout << "Ланцюг валідний? " << (chain.is_valid() ? "так" : "НІ") << "\n\n";

    // Атака 1: змінюємо дані в середині ланцюга.
    std::cout << "Підробляємо блок 1...\n";
    chain.blocks[1].data = "Аліса -> Боб: 1000";
    std::cout << "Ланцюг валідний? " << (chain.is_valid() ? "так" : "НІ") << "\n\n";

    // Атака 2: перераховуємо hash підробленого блоку, щоб "замести сліди".
    std::cout << "Перераховуємо hash блоку 1...\n";
    chain.blocks[1].hash = chain.blocks[1].calculate_hash();
    std::cout << "Ланцюг валідний? " << (chain.is_valid() ? "так" : "НІ") << "\n";
    return 0;
}
```

ТВОЄ ЗАВДАННЯ

1. Наберіть `blockchain.h` як є. `block.h` і `block.cpp` не змінюються.
2. Напишіть `blockchain.cpp` за скелетом — усі чотири методи.
3. Замініть `main.cpp`, зберіть, запустіть.

Перевірка

ПЕРЕДБАЧ ДО ЗАПУСКУ

Ключове питання розділу, дайте відповідь до запуску:

Друга атака. Зловмисник зробив усе «правильно»: змінив дані блоку 1 і оновив його хеш. Чи пройде тепер `is_valid()`? Якщо ні — на якому саме блоці вона зупиниться і з яким повідомленням?

ЕТАЛОН ДЛЯ САМОПЕРЕВІРКИ

Структура виводу (ваші хеші будуть інші):

```
#0 data="Genesis"
hash=3dcfc189650fc363... prev=0...
```

```
#1 data="Аліса -> Боб: 10"
   hash=9ba25aa27e8d68f8... prev=3dcfc189650fc363...
```

```
...
```

Ланцюг валідний? так

Підробляємо блок 1...

Ланцюг валідний? Блок 1: hash не збігається з вмістом

НІ

Перераховуємо hash блоку 1...

Ланцюг валідний? Блок 2: prev_hash не збігається з попереднім блоком

НІ

Головне, що треба побачити: після першої атаки ламається **блок 1**, після другої — **блок 2**.

Помилка перемістилась далі по ланцюгу.

САМОПЕРЕВІРКА

1. Чому `is_valid()` починає з `i = 1`, а не з `i = 0`?
2. Чому `last_block()` повертає `const Block&`, а не `Block`?
3. Що саме означає «поле `prev_hash` зчіплює блоки»? Скажіть це одним реченням.
4. Скільки блоків доведеться перерахувати зловмиснику, щоб підробка в блоці 1 ланцюга з 100 блоків стала непомітною?

Зламай навмисне

ЗЛАМАЙ НАВМИСНЕ

Експеримент 1. Приберіть другу перевірку (`prev_hash`) з `is_valid()`. Повторіть обидві атаки з `main`. Яка з них тепер проходить непоміченою?

Експеримент 2. Напишіть у `main` цикл, який після підробки блоку 1 «чесно» перераховує хеші всіх наступних блоків, і переконайтесь, що `is_valid()` знову повертає `true`. Заміряйте час — і запам'ятайте його для порівняння з наступним розділом.

Експеримент 3. У конструкторі змініть `prev_hash` генезиса з `"0"` на `"1"`. Чи стане ланцюг невалідним? Чому ні?

Експеримент 4. Додайте третю перевірку: `index` кожного блоку на 1 більший за попередній. Що зламається, якщо цю перевірку не додати, а хтось підмінить `index`?

ІДЕЯ

Після експерименту 2 стає видно межу цього кроку. Ланцюг робить підробку **помітною**, але ще не **дорогою**: перерахувати десять хешів — мікросекунди. Наступний розділ виправляє саме це.

Вправи

1. Додайте метод `size_t length() const`, що повертає кількість блоків.

2. Зробіть `print()` таким, щоб він друкував ще й `timestamp` у читабельному вигляді (підказка: `std::ctime`).

Крок 4. Proof of Work: майнінг

Мета розділу: метод `mine(difficulty)`, який робить створення блоку дорогим, і третя перевірка в `is_valid()`.

Ідея

Вимога: хеш блоку має починатися з `difficulty` нулів. Хеш непередбачуваний, тож єдиний шлях до такого хеша — міняти щось у блоці і пробувати знову. Для цього і потрібен `nonce`: збільшуємо на 1, перераховуємо хеш, дивимось.

Кожен hex-символ має 16 варіантів. Отже один нуль на початку — 1 шанс із 16, два нулі — 1 з 256, чотири — 1 з 65 536. Стільки спроб у середньому й потрібно.

ІДЕЯ

Ключова асиметрія. Знайти підходящий `nonce` дорого. Перевірити знайдений — один виклик `sha256`. Тому вся мережа може легко перевіряти роботу кожного майнера, не повторюючи її.

Це і є весь Proof of Work. Три рядки в циклі — те саме, що роблять величезні ферми ASIC-майнерів; вони лише роблять це трильйони разів на секунду.

Bitcoin порівнює хеш із числовим порогом, а не рахує нулі, і складність там не константа — вона автоматично підлаштовується, щоб блок знаходився приблизно раз на десять хвилин. Суть та сама.

Нові слова

слово	що означає
<code>std::string(n, c)</code>	Рядок із <code>n</code> копій символу: <code>std::string target(difficulty, '0')</code> дає <code>"0000"</code> .
<code>.substr(0, n)</code>	Перші <code>n</code> символів — для порівняння початку хеша з ціллю.
<code>explicit</code>	Заборона неявного перетворення: <code>Blockchain c = 4;</code> не скомпілюється, лише <code>Blockchain c(4);</code> .
<code>std::chrono</code>	Вимірювання часу: <code>steady_clock</code> , <code>duration<double></code> .

Контракт

У `Block` додається один метод:

src/block.h – доповнення

```

struct Block {
    // ... поля без змін ...

    Block(int index, const std::string& data, const std::string& prev_hash);
    std::string calculate_hash() const;

    // Підбирає nonce, поки hash не почнеться з `difficulty` нулів.
    void mine(int difficulty);
};

```

`Blockchain` отримує поле `difficulty` і конструктор з параметром:

src/blockchain.h

```

#pragma once
#include <string>
#include <vector>
#include "block.h"

class Blockchain {
public:
    explicit Blockchain(int difficulty);

    void add_block(const std::string& data);
    bool is_valid() const;
    const Block& last_block() const;
    void print() const;

    std::vector<Block> blocks;
    int difficulty;
};

```

Скелет

src/block.cpp – додати в кінець · тіла написати самому

```

void Block::mine(int difficulty) {
    // 1. Побудуйте рядок-ціль: difficulty копій символу '0'.
    // 2. Поки перші difficulty символів hash не дорівнюють цілі:
    //     - збільшити nonce на 1;
    //     - перерахувати hash через calculate_hash().
    //
    // Три рядки. Це весь майнінг.
}

```

src/blockchain.cpp – зміни · тіла написати самому

```

Blockchain::Blockchain(int difficulty) /* : difficulty(difficulty) */ {
    // Створіть генезис-блок, ЗМАЙНІТЬ його, покладіть у blocks.
}

void Blockchain::add_block(const std::string& data) {
    // Як раніше, але між створенням блоку і push_back – виклик mine(difficulty).
}

bool Blockchain::is_valid() const {
    // Дві попередні перевірки без змін, плюс третя:
    //
}

```

```
// 3. current.hash.substr(0, difficulty) == рядок з difficulty нулів
// інакше: "Блок i: hash не відповідає складності" і повернути false
//
// Підказка: рядок-ціль зручно порахувати один раз до циклу.
}

void Blockchain::print() const {
    // Додайте до виводу nonce і друкуйте хеш повністю – так видно нулі.
}
```

src/main.cpp

```
#include <chrono>
#include <iostream>
#include "blockchain.h"

int main() {
    for (int difficulty = 1; difficulty <= 5; ++difficulty) {
        auto start = std::chrono::steady_clock::now();

        Blockchain chain(difficulty);
        chain.add_block("Аліса -> Боб: 10");
        chain.add_block("Боб -> Карл: 5");

        auto end = std::chrono::steady_clock::now();
        double seconds = std::chrono::duration<double>(end - start).count();

        std::cout << "difficulty=" << difficulty
                  << " час=" << seconds << " с"
                  << " nonce останнього блоку=" << chain.last_block().nonce << "\n";
    }

    std::cout << "\nОстанній ланцюг зі складністю 4:\n";
    Blockchain chain(4);
    chain.add_block("Аліса -> Боб: 10");
    chain.print();
    std::cout << "Валідний? " << (chain.is_valid() ? "так" : "НІ") << "\n";

    chain.blocks[1].data = "Аліса -> Боб: 1000";
    chain.blocks[1].hash = chain.blocks[1].calculate_hash();
    std::cout << "Після підробки валідний? " << (chain.is_valid() ? "так" : "НІ") <<
    "\n";
    return 0;
}
```

ТВОЄ ЗАВДАННЯ

1. Додайте `mine` у `block.h` і напишіть його в `block.cpp`.
2. Перепишіть `blockchain.h` і три методи в `blockchain.cpp`.
3. Замініть `main.cpp`, зберіть, запустіть.

Перевірка

ПЕРЕДБАЧ ДО ЗАПУСКУ

До запуску заповніть таблицю очікувань — у скільки разів кожен наступний рівень складності довший за попередній, і скільки приблизно спроб потрібно для `difficulty = 4`. Потім порівняйте з реальними числами.

ЕТАЛОН ДЛЯ САМОПЕРЕВІРКИ

Ваші числа будуть іншими (комп'ютер інший, і в підборі багато випадковості), але закономірність має бути видною:

<code>difficulty=1</code>	<code>час=0.0018 с</code>	<code>nonce</code> останнього блоку=18
<code>difficulty=2</code>	<code>час=0.0032 с</code>	<code>nonce</code> останнього блоку=338
<code>difficulty=3</code>	<code>час=0.038 с</code>	<code>nonce</code> останнього блоку=1052
<code>difficulty=4</code>	<code>час=0.216 с</code>	<code>nonce</code> останнього блоку=44439
<code>difficulty=5</code>	<code>час=15.46 с</code>	<code>nonce</code> останнього блоку=4749965

Перевіряйте властивості, не числа:

- кожен хеш у виводі `print()` починається рівно з `difficulty` нулів;
- від `difficulty` 3 до 5 час росте приблизно в 16 разів на рівень (на 1–2 шум великий);
- після підробки: **Блок 1: `hash` не відповідає складності.**

Зверніть увагу, **яка саме** з трьох перевірок спіймала підробку цього разу.

САМОПЕРЕВІРКА

1. Чому підбір `nonce` дорогий, а перевірка результату — дешева?
2. Чому `mine()` міняє саме `nonce`, а не, скажімо, `timestamp`?
3. Складність 6 займе близько 40 секунд, 7 — близько десяти хвилин. Звідки ці числа?
4. У Bitcoin еквівалент — близько 19 нулів. Чому там блок все одно знаходиться раз на 10 хвилин?

Зламай навмисне

ЗЛАМАЙ НАВМИСНЕ

Експеримент 1. Повторіть експеримент 2 з Кроку 3 (чесно перемайнити весь ланцюг після підробки) зі складністю 4. Порівняйте час із тим, що ви запам'ятали. Це і є вся економіка захисту блокчейну.

Експеримент 2. Виведіть кількість хешів за секунду вашого комп'ютера (`nonce` поділити на секунди). Пошукайте, скільки робить сучасний ASIC-майнер, і порахуйте відношення.

Експеримент 3. Зробіть так, щоб `mine()` друкував крапку кожні 10 000 спроб. Запустіть зі складністю 5 — видно, як іде робота.

Експеримент 4. Приберіть третю перевірку з `is_valid()`. Створіть блок вручну, не викликаючи `mine()`, і додайте його в ланцюг із правильним `prev_hash`. Ланцюг валідний? Що це означає для мережі, де кожен може надіслати свій блок?

Крок 5. Транзакції та дерево Меркла

Мета розділу: замість рядка `data` блок несе список транзакцій; черга транзакцій (`mempool`); нагорода майнеру; всі транзакції блоку захищені одним хешем — коренем дерева Меркла.

Ідея

Транзакція — це «від кого, кому, скільки». Поле `signature` додаємо вже зараз порожнім, заповнимо в наступному розділі.

Життєвий цикл транзакції:

1. хтось створює транзакцію і кладе її в чергу `pending` (у Bitcoin це `mempool`);
2. майнер викликає `mine_pending(своя_адреса)`: бере всі транзакції з черги, додає до них нагороду собі — особливу транзакцію без відправника (`coinbase`) — і майнить блок;
3. блок стає в ланцюг, черга очищується.

Coinbase — єдиний спосіб, яким монети з'являються у світі. Так само в Bitcoin: кожен блок створює нові біткоїни і віддає їх майнеру.

Дерево Меркла

Як включити список транзакцій у хеш блоку? Можна склеїти їх усі в рядок і хешувати — працює. Bitcoin робить інакше: хеші транзакцій попарно об'єднують і хешують, отримані хеші знову попарно, і так до одного — **кореня**.

структура

```
          root = H(h01 + h23)
        /      \
    h01 = H(h0 + h1)  h23 = H(h2 + h3)
    /  \          /  \
h0=H(tx0) h1=H(tx1) h2=H(tx2) h3=H(tx3)
```

Якщо кількість непарна, останній хеш дублюють. Корінь — один рядок, який змінюється від зміни будь-якої транзакції. Саме він іде в хеш блоку.

ЧОМУ САМЕ ТАК

Навіщо дерево, якщо можна просто склеїти? Бо з деревом можна довести, що транзакція `tx2` входить у блок, показавши лише `h3`, `h01` і корінь — три хеші замість усіх тисяч транзакцій блоку. Так працюють «легкі» гаманці на телефонах: вони не зберігають блоки, лише заголовки з коренями. Ми цим не скористаємось, але структура буде правильна.

Нові слова

слово	що означає
<code>std::vector<Transaction></code>	Список транзакцій усередині блоку.
<code>Transaction{a, b, c, d}</code>	Створення структури значеннями полів по порядку — §0.11.
<code>Block genesis(0, {}, "0")</code>	Порожні фігурні дужки = порожній вектор.
<code>::merkle_root(...)</code>	Подвійна двокрапка на початку = «глобальна функція», а не однойменне поле. Критично в списку ініціалізації.
<code>static const long long</code>	Константа, спільна для всього класу — §0.19.
<code>level.size() % 2 == 1</code>	Перевірка на непарність.
<code>.clear()</code>	Спорожнити вектор.
<code>.empty()</code>	Чи порожній.

Контракт

src/transaction.h

```
#pragma once
#include <string>

struct Transaction {
    std::string from;           // адреса відправника; "" = нагорода майнеру (coinbase)
    std::string to;             // адреса отримувача
    long long amount;           // сума в монетах (ціле число, НЕ double)
    std::string signature;      // підпис – поки що порожній, додамо в наступному розділі

    // Рядок з полів, які захищає підпис (signature сюди НЕ входить).
    std::string to_string() const;

    // Хеш транзакції = її унікальний ідентифікатор.
    std::string hash() const;
};
```

src/merkle.h

```
#pragma once
#include <string>
#include <vector>
#include "transaction.h"

// Обчислює корінь дерева Меркла для списку транзакцій.
std::string merkle_root(const std::vector<Transaction>& transactions);
```

src/block.h

```
#pragma once
#include <string>
#include <vector>
#include "transaction.h"

struct Block {
    int index;
    long timestamp;
    std::vector<Transaction> transactions; // замість рядка data
};
```

```

std::string merkle_root;           // "відбиток" усіх транзакцій
std::string prev_hash;
long nonce;
std::string hash;

Block(int index, const std::vector<Transaction>& transactions,
      const std::string& prev_hash);

std::string calculate_hash() const;
void mine(int difficulty);
};

```

src/blockchain.h

```

#pragma once
#include <string>
#include <vector>
#include "block.h"
#include "transaction.h"

class Blockchain {
public:
    explicit Blockchain(int difficulty);

    // Кладе транзакцію в "чергу" (мемпул). У блок вона потрапить після майнінгу.
    void add_transaction(const Transaction& tx);

    // Збирає всі транзакції з черги в новий блок і майнить його.
    // Майнер отримує нагороду MINING_REWARD.
    void mine_pending(const std::string& miner_address);

    bool is_valid() const;
    const Block& last_block() const;
    void print() const;

    std::vector<Block> blocks;
    std::vector<Transaction> pending;
    int difficulty;

    static const long long MINING_REWARD = 50;
};

```

Скелет

src/transaction.cpp · тіла написати самому

```

#include "transaction.h"
#include "sha256.h"

std::string Transaction::to_string() const {
    // Склейте: from + "->" + to + ":" + amount (число через std::to_string).
    // signature сюди НЕ входить – інакше вийде замкнене коло:
    // підпис від рядка, що містить підпис.
}

std::string Transaction::hash() const {
    // sha256 від to_string().
}

```

src/merkle.cpp · тіла написати самому

```

#include "merkle.h"
#include "sha256.h"

std::string merkle_root(const std::vector<Transaction>& transactions) {
    // 1. Порожній список -> поверніть sha256("") (фіксоване значення для генезиса).
    //
    // 2. Рівень 0: заведіть std::vector<std::string> level
    //    і покладіть у нього хеш кожної транзакції.
    //
    // 3. Поки в level більше одного елемента:
    //    - якщо кількість непарна, продублюйте останній елемент;
    //    - зберіть новий вектор `next`: для i = 0, 2, 4, ...
    //      покладіть sha256(level[i] + level[i+1]);
    //    - замініть level на next.
    //
    // 4. Поверніть єдиний елемент, що лишився.
}

```

src/block.cpp – зміни · тіла написати самому

```

Block::Block(int index, const std::vector<Transaction>& transactions,
              const std::string& prev_hash)
    // Список ініціалізації як раніше, але:
    // - замість data тепер transactions(transactions);
    // - додається merkle_root(merkle_root(transactions)).
    // Подвійна двокрапка обов'язкова – інакше компілятор візьме поле,
    // яке в цей момент ще не ініціалізоване.
{
    // hash = calculate_hash();
}

std::string Block::calculate_hash() const {
    // Те саме, що раніше, але замість data – merkle_root.
    // Усі транзакції захищені через один рядок.
}

```

src/blockchain.cpp – зміни · тіла написати самому

```

Blockchain::Blockchain(int difficulty) /* : difficulty(difficulty) */ {
    // Генезис тепер із порожнім списком транзакцій: Block genesis(0, {}, "0");
}

void Blockchain::add_transaction(const Transaction& tx) {
    // Покласти в pending.
}

void Blockchain::mine_pending(const std::string& miner_address) {
    // 1. Зробіть транзакцію-нагороду: from порожній, to = miner_address,
    //    amount = MINING_REWARD, signature порожній.
    // 2. Скопіюйте pending у новий вектор і додайте туди нагороду.
    // 3. Створіть блок з цим вектором, змайніть, покладіть у blocks.
    // 4. Очистіть pending.
}

bool Blockchain::is_valid() const {
    // Три попередні перевірки без змін, плюс четверта:
    //
    // 4. current.merkle_root == merkle_root(current.transactions)
    //    інакше: "Блок i: транзакції змінено (merkle root не збігається)"
}

```

```
void Blockchain::print() const {
    // Для кожного блоку: номер, попси, перші 16 символів хеша,
    // і під ним – рядок на кожну транзакцію.
    // Порожній from друкуйте як "[нагорода]".
}
```

src/main.cpp

```
#include <iostream>
#include "blockchain.h"

int main() {
    Blockchain chain(3);

    chain.add_transaction({"alice", "bob", 10, ""});
    chain.add_transaction({"bob", "carol", 5, ""});
    chain.mine_pending("miner");

    chain.add_transaction({"carol", "alice", 2, ""});
    chain.mine_pending("miner");

    chain.print();
    std::cout << "Валідний? " << (chain.is_valid() ? "так" : "НІ") << "\n\n";

    // Атака: міняємо суму в уже змайненому блоці.
    chain.blocks[1].transactions[0].amount = 1000;
    std::cout << "Після підробки валідний? " << (chain.is_valid() ? "так" : "НІ") <<
    "\n";
    return 0;
}
```

ТВОЄ ЗАВДАННЯ

1. Створіть `transaction.h/.cpp` і `merkle.h/.cpp`.
2. Перепишіть `block.h`, `block.cpp`, `blockchain.h`, `blockchain.cpp` за скелетами.
3. Замініть `main.cpp`, зберіть, запустіть.

Перевірка

ПЕРЕДБАЧ ДО ЗАПУСКУ

Перед запуском: скільки транзакцій буде в блоці 1, якщо ви поклали в чергу дві? А в блоці 2, якщо одну? Чому?

ЕТАЛОН ДЛЯ САМОПЕРЕВІРКИ

```
#0 nonce=326 hash=00088b0ed2acbe7d...
#1 nonce=12705 hash=000c588333ba22ee...
  alice -> bob : 10
  bob -> carol : 5
  [нагорода] -> miner : 50
#2 nonce=52 hash=000c531e542e136d...
  carol -> alice : 2
  [нагорода] -> miner : 50
Валідний? так
```

Після підробки валідний? Блок 1: транзакції змінено (merkle root не збігається)
НІ

Властивості для перевірки:

- у блоці 1 три транзакції (дві ваші + нагорода), у 2 дві;
- у генезисі транзакцій немає;
- підробку ловить саме четверта перевірка.

САМОПЕРЕВІРКА

1. Чому `signature` не входить у `to_string()` ?
2. Що поверне `merkle_root` для порожнього списку і навіщо це потрібно?
3. Навіщо `::u::merkle_root(transactions)` всередині списку ініціалізації?
4. Чому підробку спіймала четверта перевірка, а не перша (`hash` проти вмісту)?

Зламай навмисне

ЗЛАМАЙ НАВМИСНЕ

Експеримент 1. Заберіть четверту перевірку з `is_valid()` і повторіть атаку з `main`. Ланцюг її більше не бачить — поясніть точно чому. (Підказка: що входить у `calculate_hash()`, а що ні.) Поверніть перевірку.

Експеримент 2. Приберіть `::u` у списку ініціалізації `block.cpp`. Що скаже компілятор? А якщо він промовчить — що буде в `merkle_root` під час виконання?

Експеримент 3. Змайніть блок з 5 транзакціями і з 6. Намалуйте на папері дерево Меркла для кожного випадку: де саме дублюється останній хеш і скільки разів?

Експеримент 4. Напишіть функцію, що рахує загальну кількість монет у ланцюгу (суму всіх `coinbase`). Переконайтесь, що вона дорівнює $50 \times (\text{кількість блоків} - 1)$.

ІДЕЯ

Наша монета вже вміє те, що робить будь-яка криптовалюта: збирати перекази в блоки і винагороджувати майнерів. Але зараз будь-хто може написати `{"alice", "bob", 1000, ""}` — і ланцюг радісно це прийме. Аліса нічого не підписувала. Виправляємо.

Крок 6. Гаманець і цифровий підпис

Мета розділу: клас `Wallet` з парою ключів; підписання транзакції приватним ключем і перевірка публічним.

Це найважчий розділ книги з точки зору API, але не з точки зору ідеї. Закладіть на нього два сеанси і не лякайтесь довжини `wallet.cpp`.

Ідея

У гаманця дві частини:

- **приватний ключ** — випадкове 256-бітне число, яке знає лише власник;
- **публічний ключ** — точка на еліптичній кривій, обчислена з приватного. З публічного відновити приватний неможливо.

У нашій монеті **адреса** — це публічний ключ у hex (130 символів). Довго і некрасиво, зате чесно: щоб перевірити підпис, потрібен саме публічний ключ, і він завжди під рукою.

Три публічні операції, і тільки вони мають бути зрозумілі:

1. `address()` — моя адреса (публічний ключ у hex);
2. `sign(повідомлення)` — рядок, який залежить від повідомлення і приватного ключа;
3. `verify_signature(адреса, повідомлення, підпис)` — `true` лише якщо підпис зроблено приватним ключем, що відповідає адресі, і повідомлення не змінилося.

Підписуємо ми хеш транзакції — `tx.hash()`. Підпис зберігаємо в полі `signature`.

Крива — `secp256k1`, та сама, що в Bitcoin та Ethereum. Алгоритм — ECDSA. Усе реалізує OpenSSL; ми лише правильно його викликаємо.

УВАГА

Ніколи не реалізуйте криптографію самостійно для реальних грошей. Навіть правильно написаний алгоритм ламають через час виконання, кеш процесора чи погану випадковість. Bitcoin Core використовує окрему бібліотеку `libsecp256k1`, вилізана роками. Для навчання OpenSSL більш ніж достатньо.

Нові слова

слово	що означає
<code>struct evp_pkey_st;</code>	Попереднє оголошення: дозволяє тримати вказівник на тип, не підключаючи його заголовок.
<code>~Wallet()</code>	Деструктор — виконується при знищенні об'єкта. Тут звільняє ключ.

слово	що означає
<code>= delete</code>	Заборонити операцію. <code>Wallet(const Wallet&) = delete;</code> забороняє копіювання.
<code>private:</code>	Далі — те, що видно лише методам класу — §0.12.
<code>EVP_PKEY*</code>	Вказівник на об'єкт ключа OpenSSL.
<code>static (y .cpp)</code>	Функція видима лише в цьому файлі — §0.19.
<code>throw</code>	Кинути виняток — §0.19.
<code>std::stoi(s, nullptr, 16)</code>	Рядок як число в основі 16.
<code>vector.resize(n)</code>	Змінити розмір вектора.
<code>class Wallet;</code>	Попереднє оголошення класу: <code>transaction.h</code> знає лише ім'я, а не вміст.

Контракт

src/wallet.h

```
#pragma once
#include <string>

// Попереднє оголошення, щоб не тягнути заголовки OpenSSL сюди.
struct evp_pkey_st;
typedef evp_pkey_st EVP_PKEY;

class Wallet {
public:
    Wallet();    // генерує нову пару ключів (крива secp256k1, як у Bitcoin)
    ~Wallet();

    // Копіювати гаманець не можна (всередині сирий вказівник OpenSSL).
    Wallet(const Wallet&) = delete;
    Wallet& operator=(const Wallet&) = delete;

    // Публічний ключ у hex. Це і є "адреса" в нашій монеті.
    std::string address() const;

    // Підписує повідомлення приватним ключем. Повертає підпис у hex.
    std::string sign(const std::string& message) const;

private:
    EVP_PKEY* key_;
};

// Перевіряє підпис: чи справді власник `address` підписав `message`?
bool verify_signature(const std::string& address,
                     const std::string& message,
                     const std::string& signature_hex);
```

ЧОМУ САМЕ ТАК

Чому `= delete` для копіювання? Якщо скопіювати `Wallet`, обидві копії триматимуть один вказівник, і обидва деструктори спробують його звільнити. Друге звільнення — падіння програми. Забороняємо копіювання, і компілятор не дасть зробити це випадково.

Чому **verify_signature** — вільна функція, а не метод? Для перевірки не потрібен гаманець, лише адреса. Метод вимагав би об'єкта, якого немає.

src/transaction.h

```
#pragma once
#include <string>

class Wallet;

struct Transaction {
    std::string from;
    std::string to;
    long long amount;
    std::string signature;

    std::string to_string() const;
    std::string hash() const;

    // Підписує транзакцію гаманцем відправника.
    void sign(const Wallet& wallet);

    // Нагорода майнеру — завжди ок; інакше підпис має збігатися з from.
    bool is_valid() const;
};
```

Скелет

wallet.cpp — найдовший файл книги. Він поділений на шість частин; пишіть їх по черзі і збирайте після кожної.

src/wallet.cpp · тіла написати самому

```
#include "wallet.h"

#include <openssl/core_names.h>
#include <openssl/evp.h>
#include <openssl/param_build.h>

#include <iomanip>
#include <sstream>
#include <stdexcept>
#include <vector>

// ----- частина 1: байти <=> hex -----

static std::string to_hex(const std::vector<unsigned char>& bytes) {
    // Те саме, що в кінці sha256(): ostringstream, hex, setw(2), setfill('0').
}

static std::vector<unsigned char> from_hex(const std::string& hex) {
    // Зворотне: берить по два символи і перетворює через
    // std::stoi(hex.substr(i, 2), nullptr, 16).
}

// ----- частина 2: конструктор і деструктор -----

Wallet::Wallet() {
```

```

    // key_ = EVP_PKEY_Q_keygen(nullptr, nullptr, "EC", "secp256k1");
    // Якщо повернувся nullptr – киньте std::runtime_error.
}

Wallet::~Wallet() {
    // Звільніть key_.
}

// ----- частина 3: адреса -----

std::string Wallet::address() const {
    // 1. Буфер unsigned char buf[128] і size_t len = 0.
    // 2. EVP_PKEY_get_octet_string_param(key_, OSSLPKEY_PARAM_PUB_KEY,
    //                                     buf, sizeof(buf), &len)
    // 3. Перетворіть перші len байтів у hex через to_hex.
    //    Для secp256k1 це 65 байтів: 0x04 плюс дві координати по 32.
}

// ----- частина 4: підпис -----

std::string Wallet::sign(const std::string& message) const {
    // 1. Створіть EVP_MD_CTX, викличте EVP_DigestSignInit(ctx, nullptr,
    //     EVP_sha256(), nullptr, key_).
    // 2. Викличте EVP_DigestSign ДВІчі:
    //     перший раз з nullptr замість буфера – щоб дізнатися розмір sig_len;
    //     другий – уже з буфером потрібного розміру.
    // 3. Після другого виклику зробіть sig.resize(sig_len):
    //     підписи ECDSA змінної довжини, і реальна може бути меншою за обіцяну.
    // 4. Звільніть контекст, поверніть to_hex(sig).
}

// ----- частина 5: зібрати публічний ключ з байтів -----

static EVP_PKEY* load_public_key(const std::vector<unsigned char>& pub) {
    // Операція, зворотна до address(). OpenSSL 3 робить це через "параметри":
    // 1. OSSL_PARAM_BLD_new()
    // 2. push_utf8_string(bld, OSSLPKEY_PARAM_GROUP_NAME, "secp256k1", 0)
    // 3. push_octet_string(bld, OSSLPKEY_PARAM_PUB_KEY, pub.data(), pub.size())
    // 4. OSSL_PARAM_BLD_to_param(bld) -> params
    // 5. EVP_PKEY_CTX_new_from_name(nullptr, "EC", nullptr),
    //     EVP_PKEY_fromdata_init, EVP_PKEY_fromdata(ctx, &key, EVP_PKEY_PUBLIC_KEY,
    params)
    // 6. Звільніть усе створене і поверніть key.
    //     Якщо байти не є коректною точкою кривої – там буде nullptr.
}

// ----- частина 6: перевірка -----

bool verify_signature(const std::string& address,
                     const std::string& message,
                     const std::string& signature_hex) {
    // 1. Зберіть ключ із адреси через load_public_key(from_hex(address)).
    //     nullptr -> одразу false.
    // 2. Дзеркало sign(): EVP_DigestVerifyInit, потім EVP_DigestVerify.
    // 3. Результат 1 означає "підпис вірний". Звільніть усе створене.
}

```

src/transaction.cpp – доповнення · тіла написати самому

```

void Transaction::sign(const Wallet& wallet) {
    // signature = підпис від hash() гаманцем.

```

```

}

bool Transaction::is_valid() const {
    // 1. Порожній from -> це coinbase, підпису не потребує -> true.
    // 2. Порожній signature -> false.
    // 3. Інакше -> verify_signature(from, hash(), signature).
}

```

src/main.cpp

```

#include <iostream>
#include "transaction.h"
#include "wallet.h"

int main() {
    Wallet alice;
    Wallet bob;

    std::cout << "Адреса Аліси: " << alice.address().substr(0, 20) << "...\\n";
    std::cout << "Адреса Боба: " << bob.address().substr(0, 20) << "...\\n\\n";

    // Аліса створює і підписує транзакцію.
    Transaction tx{alice.address(), bob.address(), 10, ""};
    tx.sign(alice);
    std::cout << "Підпис: " << tx.signature.substr(0, 20) << "...\\n";
    std::cout << "Транзакція валідна? " << (tx.is_valid() ? "так" : "НІ") << "\\n\\n";

    // Атака 1: Боб змінює суму після підпису.
    tx.amount = 1000;
    std::cout << "Змінили суму на 1000. Валідна? " << (tx.is_valid() ? "так" : "НІ") << "\\n";
    tx.amount = 10;

    // Атака 2: Боб намагається підписати транзакцію "від Аліси" своїм ключем.
    Transaction fake{alice.address(), bob.address(), 10, ""};
    fake.sign(bob);
    std::cout << "Підписано ключем Боба. Валідна? " << (fake.is_valid() ? "так" : "НІ") << "\\n";

    // Атака 3: без підпису взагалі.
    Transaction unsigned_tx{alice.address(), bob.address(), 10, ""};
    std::cout << "Без підпису. Валідна? " << (unsigned_tx.is_valid() ? "так" : "НІ") << "\\n";
    return 0;
}

```

УВАГА

Файли `block.*`, `merkle.*`, `blockchain.*` у цьому розділі не використовуються в `main`, але залишаються в проєкті і мають збиратись. Не видаляйте їх.

ТВОЄ ЗАВДАННЯ

1. Наберіть `wallet.h` як є — саме він визначає, що ви робите.
2. Напишіть `wallet.cpp` частинами 1–6, збираючи після кожної.
3. Додайте `sign` і `is_valid` у `transaction.h` та `transaction.cpp`.
4. Замініть `main.cpp`, зберіть, запустіть.

Цей файл — «чорна скринька», і це нормально. Абсолютно припустимо не розуміти кожен виклик OpenSSL: він написаний на С у стилі 90-х, з довгими іменами і ручним керуванням пам'яттю. Важливо розуміти, **що роблять три публічні функції**. Решта — обгортка.

Перевірка

ПЕРЕДБАЧ ДО ЗАПУСКУ

Три атаки в `main` мають дати три відповіді. Запишіть свої передбачення для кожної і **причину**, чому саме така:

1. зміна суми після підпису;
2. підпис чужим ключем;
3. відсутність підпису.

ЕТАЛОН ДЛЯ САМОПЕРЕВІРКИ

Адреса Аліси: 0483973267e8318f67df...

Адреса Боба: 0455fd01a7f30bca07cc...

Підпис: 3046022100b252df90e7...

Транзакція валідна? так

Змінили суму на 1000. Валідна? НІ

Підписано ключем Боба. Валідна? НІ

Без підпису. Валідна? НІ

При кожному запуску ключі нові, тому адреси й підписи будуть інші. Незмінне:

- адреса має рівно 130 символів і починається з **04** — маркер нестисненої точки кривої;
- підпис починається з **30** — це DER-кодування, стандартний формат ECDSA;
- три «НІ» у трьох атаках.

ІДЕЯ

Ці три «НІ» — три властивості, на яких тримається будь-яка криптовалюта: **підписане не можна змінити, за іншого не можна підписати, без підпису не рахується**.

САМОПЕРЕВІРКА

1. Чому копіювання `Wallet` заборонено, а не просто «не рекомендовано»?
2. Чому `EVP_DigestSign` викликається двічі?
3. Що станеться, якщо у `verify_signature` передати замість адреси рядок `"abc"`?
4. Чому `transaction.h` пише `class Wallet;` замість `#include "wallet.h"`?
5. Чому coinbase-транзакція валідна без підпису — і хто заважає підробити її?

Зламай навмисне

ЗЛАМАЙ НАВМИСНЕ

Експеримент 1. Підпишіть один і той самий рядок двічі тим самим гаманцем. Підписи однакові? (ECDSA використовує випадкове число при підписанні.) Обидва валідні? Перевірте.

Експеримент 2. Передайте у `verify_signature` замість адреси рядок `"abc"`. Програма падає чи повертає `false`? Який рядок коду це забезпечує?

Експеримент 3. Приберіть `sig.resize(sig_len)` після другого `EVP_DigestSign`. Перевірка почне іноді провалюватись — запустіть кілька разів. Чому «іноді», а не завжди?

Експеримент 4. Приберіть `= delete` для копіювання і напишіть `Wallet b = a;`. Що станеться при виході з області видимості? (Готуйтеся побачити падіння.)

Вправи

1. Зробіть у `Transaction` метод `short_from()`, що повертає перші 10 символів адреси — стане в пригоді для виводу.
2. Додайте у `Wallet` метод `public_key_bytes()`, що повертає `std::vector<unsigned char>` замість hex-рядка.

Крок 7. Баланси і правила монети

Мета розділу: `balance_of(адреса)`; `add_transaction` відхиляє невалідні транзакції і витрати понад баланс; `is_valid()` перевіряє підписи в блоках. Після цього розділу ланцюг стає повноцінною (хоч і іграшковою) монетою.

Ідея

У нашій монеті немає окремої таблиці балансів. **Баланс адреси** — це сума всього, що їй прийшло, мінус усе, що від неї пішло, по всіх блоках ланцюга. Ланцюг — єдине джерело правди; баланс завжди можна перерахувати з нуля.

ІДЕЯ

Дрібниця, яка ловить подвійну витрату. Рахуючи баланс, треба враховувати ще й транзакції з черги `pending`. Інакше Боб із 10 монетами міг би покласти в чергу дві транзакції по 10 — кожна окремо проходить перевірку проти ланцюга, де в нього ще 10.

Правила прийому транзакції в чергу, у такому порядку:

1. це не `coinbase` (нагороду можна створити лише через `mine_pending`);
2. сума додатна (інакше «переказ -50» — це крадіжка);
3. підпис вірний;
4. баланс вистачає.

ЧОМУ САМЕ ТАК

Порядок навмисний: спершу дешеві перевірки (порожній `from`, сума), далі дорога (перевірка підпису — еліптична криптографія), і найдорожча — баланс, який вимагає прогону по всьому ланцюгу — останньою. Так більшість сміття відсіюється миттєво.

І п'ята перевірка в `is_valid()`: кожна транзакція в кожному блоці має вірний підпис. Без неї зловмисник міг би змайнити блок із непідписаною транзакцією «від Аліси» — майнер сам собі суддя.

Це **модель рахунків**, як в Ethereum. Bitcoin працює інакше — за моделлю UTXO, де немає балансів, а є «невитрачені виходи» попередніх транзакцій, які нова транзакція витрачає цілком. UTXO складніша для початківця, тому тут модель рахунків.

Нові слова

Нових конструкцій мови в цьому розділі немає — усе вже знайоме. Розділ про правила, а не про синтаксис. Єдина зміна сигнатури:

слово	що означає
<code>bool add_transaction(...)</code>	Метод тепер повертає результат, а не <code>void</code> : викликач має знати, прийняли транзакцію чи ні.

Контракт

src/blockchain.h

```
#pragma once
#include <string>
#include <vector>
#include "block.h"
#include "transaction.h"

class Blockchain {
public:
    explicit Blockchain(int difficulty);

    // Повертає false і НЕ додає транзакцію, якщо вона невалідна
    // або в відправника недостатньо монет.
    bool add_transaction(const Transaction& tx);

    void mine_pending(const std::string& miner_address);

    // Скільки монет у адреси: сума всіх вхідних мінус усіх вихідних транзакцій.
    long long balance_of(const std::string& address) const;

    bool is_valid() const;
    const Block& last_block() const;
    void print() const;

    std::vector<Block> blocks;
    std::vector<Transaction> pending;
    int difficulty;

    static const long long MINING_REWARD = 50;
};
```

Скелет

src/blockchain.cpp – зміни · тіла написати самому

```
long long Blockchain::balance_of(const std::string& address) const {
    // 1. Заведіть long long balance = 0.
    // 2. Подвійний цикл: по всіх блоках, у кожному – по всіх транзакціях.
    //     якщо tx.to == address -> додати amount
    //     якщо tx.from == address -> відняти amount
    // 3. Окремий цикл по pending: усе, що ця адреса вже "обіцяла",
    //     теж віднімається. Саме це ловить подвійну витрату.
}

bool Blockchain::add_transaction(const Transaction& tx) {
    // Чотири перевірки в порядку зростання вартості. Кожна невдача:
    // надрукувати причину і повернути false.
    //
    // 1. tx.from порожній -> "Відхилено: нагороду майнеру не можна додати вручну"
```

```

// 2. tx.amount <= 0    -> "Відхилено: сума має бути додатною"
// 3. !tx.is_valid()    -> "Відхилено: невірний підпис"
// 4. balance_of(tx.from) < tx.amount
//                        -> "Відхилено: недостатньо монет (баланс X, потрібно Y)"
//
// Усе пройшло – покласти в pending і повернути true.
}

bool Blockchain::is_valid() const {
    // Чотири попередні перевірки без змін, плюс п'ята:
    //
    // 5. для кожної транзакції в блоці: tx.is_valid()
    //     інакше "Блок i: транзакція з невірним підписом" і false
}

void Blockchain::print() const {
    // Адреси тепер по 130 символів – друкуйте лише перші 10 і "...".
}

```

src/main.cpp

```

#include <iostream>
#include "blockchain.h"
#include "wallet.h"

int main() {
    Blockchain chain(3);
    Wallet alice;
    Wallet bob;

    std::cout << "---- Аліса майнить перший блок ----\n";
    chain.mine_pending(alice.address());
    std::cout << "Баланс Аліси: " << chain.balance_of(alice.address()) << "\n";
    std::cout << "Баланс Боба: " << chain.balance_of(bob.address()) << "\n\n";

    std::cout << "---- Аліса надсилає Бобу 10 ----\n";
    Transaction tx{alice.address(), bob.address(), 10, ""};
    tx.sign(alice);
    chain.add_transaction(tx);
    chain.mine_pending(alice.address()); // Аліса майнить і отримує ще 50
    std::cout << "Баланс Аліси: " << chain.balance_of(alice.address()) << "\n";
    std::cout << "Баланс Боба: " << chain.balance_of(bob.address()) << "\n\n";

    std::cout << "---- Боб намагається надіслати 1000 ----\n";
    Transaction too_much{bob.address(), alice.address(), 1000, ""};
    too_much.sign(bob);
    chain.add_transaction(too_much);

    std::cout << "\n---- Боб підробляє транзакцію від імені Аліси ----\n";
    Transaction forged{alice.address(), bob.address(), 50, ""};
    forged.sign(bob); // у Боба немає ключа Аліси
    chain.add_transaction(forged);

    std::cout << "\n---- Подвійна витрата: Боб двічі надсилає свої 10 ----\n";
    Transaction first{bob.address(), alice.address(), 10, ""};
    first.sign(bob);
    Transaction second{bob.address(), alice.address(), 10, ""};
    second.sign(bob);
    std::cout << "Перша: " << (chain.add_transaction(first) ? "прийнято" :
"Відхилено") << "\n";
    std::cout << "Друга: " << (chain.add_transaction(second) ? "прийнято" :

```

```

"Відхилено") << "\n";
chain.mine_pending(bob.address());

std::cout << "\n";
chain.print();
std::cout << "\nВалідний? " << (chain.is_valid() ? "так" : "НІ") << "\n";
return 0;
}

```

ТВОЄ ЗАВДАННЯ

1. Оновіть `blockchain.h`.
2. Напишіть `balance_of`, перепишіть `add_transaction`, додайте п'яту перевірку.
3. Замініть `main.cpp`, зберіть, запустіть.

Перевірка

ПЕРЕДБАЧ ДО ЗАПУСКУ

Найважливіше передбачення книги. До запуску порахуйте на папері:

1. Аліса змайнила блок. Її баланс?
2. Аліса надіслала Бобу 10 і змайнила ще блок. Баланс Аліси? Баланс Боба?
3. Боб кладе в чергу дві транзакції по 10, маючи 10. Яка з них пройде і чому?
4. Скільки всього монет у світі після трьох змайнених блоків?

ЕТАЛОН ДЛЯ САМОПЕРЕВІРКИ

--- Аліса майнить перший блок ---

Баланс Аліси: 50

Баланс Боба: 0

--- Аліса надсилає Бобу 10 ---

Баланс Аліси: 90

Баланс Боба: 10

--- Боб намагається надіслати 1000 ---

Відхилено: недостатньо монет (баланс 10, потрібно 1000)

--- Боб підробляє транзакцію від імені Аліси ---

Відхилено: невірний підпис

--- Подвійна витрата: Боб двічі надсилає свої 10 ---

Перша: прийнято

Друга: Відхилено: недостатньо монет (баланс 0, потрібно 10)

відхилено

Простежте за балансами: Аліса змайнила два блоки (100), віддала 10 і отримала 10 назад — 90 плюс 10 у черзі. Боб отримав 10, віддав 10, змайнив блок — 50. Загалом у світі 150 монет = 3 блоки × 50.

Монети нізвідки не беруться і нікуди не зникають. Це і є головна властивість, яку має мати монета.

САМОПЕРЕВІРКА

1. Чому `balance_of` враховує `pending`, а не лише блоки?
2. Чому перевірка балансу стоїть останньою, а не першою?
3. Що станеться, якщо прибрати п'яту перевірку з `is_valid()`, і хто цим скористається?
4. Звідки взялась цифра 150 і чому вона не може бути іншою?

Зламай навмисне

ЗЛАМАЙ НАВМИСНЕ

Експеримент 1. Приберіть цикл по `pending` з `balance_of`. Повторіть сценарій подвійної витрати. Обидві транзакції проходять? Скільки монет тепер «створив» Боб із повітря?

Експеримент 2. Приберіть перевірку `tx.amount <= 0` і надішліть транзакцію на `-50`. Що станеться з балансами відправника й отримувача?

Експеримент 3. Додайте перевірку, що `from != to` — переказ самому собі безглуздий, але зараз проходить. Що він робить із балансом і чому?

Експеримент 4. Зробіть `MINING_REWARD` залежним від номера блоку: 50 для перших 10 блоків, 25 для наступних 10, і так далі. Це **халвінг** — механізм, через який біткоїнів ніколи не буде більше 21 мільйона. Порахуйте, до якої суми сходиться ваш ряд.

Вправи

1. Напишіть `total_supply()` — суму всіх монет в обігу — і переконайтесь, що вона дорівнює сумі всіх балансів.
2. Додайте комісію: майнер отримує `MINING_REWARD` плюс 1% від кожної транзакції в блоці.

Крок 8. Збереження на диск

Мета розділу: ланцюг і ключ гаманця переживають перезапуск програми. Заразом — перший досвід **серіалізації**: перетворення об'єктів у текст і назад.

Ідея

Найпростіший формат, який працює: один блок — один рядок текстового файлу, поля через `|`, транзакції через `;`, поля транзакції через `:`:

`chain.txt`

```
0|1788999378|0|2610|0001a97b09a2...|
1|1788999378|0001a97b09a2...|4452|000a86cb4bee...|,04ca01...,50,
2|1788999378|000a86cb4bee...|1251|000f65eb9a89...|
04ca01...,04d0cb...,10,3046...; ,04ca01...,50,
```

Роздільники можна брати будь-які, головне — щоб вони не зустрічались у даних. У нас усі поля — числа або hex-рядки, тож `|`, `;` і `,` безпечні.

ІДЕЯ

Ключовий момент при читанні: ми не довіряємо збереженому **hash**. Блок відновлюється з полів, хеш перераховується і порівнюється зі збереженим. Якщо хтось відредагував файл руками — завантаження провалиться. Ланцюг захищає себе навіть на диску.

Приватний ключ зберігаємо у форматі PEM — текстовий стандарт, який OpenSSL пише й читає однією функцією.

ЧОМУ САМЕ ТАК

Функції розділені на дві пари: «текст ↔ ланцюг» (`serialize_chain` / `deserialize_chain`) і «файл ↔ текст» (`save_chain` / `load_chain`). Це не надмірність: у наступному розділі той самий текст піде не у файл, а в мережу, і перша пара запрацює без жодної зміни.

Нові слова

слово	що означає
делегування конструктора	<code>Block::Block(a, b) : Block(a, час, b, 0) {}</code> — один конструктор викликає інший, щоб не дублювати список ініціалізації.
<code>std::ofstream</code> / <code>ifstream</code>	Файл на запис / на читання — §0.17.
<code>std::getline(in, line)</code>	Прочитати рядок до <code>\n</code> . Повертає <code>false</code> , коли рядки скінчились.
<code>std::istreamstream</code>	Рядок, з якого можна читати як з файлу — §0.17.
<code>buffer << file.rdbuf()</code>	Прочитати весь файл одним рухом.
<code>std::stoi</code> / <code>stol</code> / <code>stoll</code>	Рядок у <code>int</code> / <code>long</code> / <code>long long</code> .

слово	що означає
FILE*, fopen, fclose	Файли в стилі C. Потрібні, бо саме їх хоче OpenSSL.
PEM_write_PrivateKey	Записати приватний ключ у текстовому форматі PEM.

Контракт

Block отримує другий конструктор — для відновлення з відомими **timestamp** і **nonce**:

src/block.h — доповнення

```
// Новий блок (timestamp = зараз, nonce = 0).
Block(int index, const std::vector<Transaction>& transactions,
      const std::string& prev_hash);

// Відновлення блоку з диска/мережі: усі поля відомі, hash перераховується.
Block(int index, long timestamp, const std::vector<Transaction>& transactions,
      const std::string& prev_hash, long nonce);
```

src/storage.h

```
#pragma once
#include <string>
#include "blockchain.h"

// Ланцюг -> текст. Один блок на рядок:
//   index|timestamp|prev_hash|nonce|hash|tx1;tx2;...
// де кожна транзакція: from,to,amount,signature
std::string serialize_chain(const Blockchain& chain);

// Текст -> ланцюг. Повертає false, якщо формат зламано або hash не сходиться.
bool deserialize_chain(const std::string& text, Blockchain& chain);

bool save_chain(const Blockchain& chain, const std::string& path);
bool load_chain(Blockchain& chain, const std::string& path);
```

Wallet отримує два методи:

src/wallet.h — доповнення

```
// Зберігає приватний ключ у файл (формат PEM). НИКОМУ не показуйте цей файл.
bool save(const std::string& path) const;

// Завантажує ключ з файлу замість щойно згенерованого.
bool load(const std::string& path);
```

Скелет

src/block.cpp — зміни · тіла написати самому

```
Block::Block(int index, const std::vector<Transaction>& transactions,
             const std::string& prev_hash)
    // Делегування: викличте другий конструктор із std::time(nullptr) і nonce = 0.
    // Тіло порожнє.

Block::Block(int index, long timestamp, const std::vector<Transaction>& transactions,
             const std::string& prev_hash, long nonce)
    // Повний список ініціалізації, як був, але timestamp і nonce беруться з
```

```

    параметрів.
    {
        // hash = calculate_hash();
    }

```

src/storage.cpp · тіла написати самому

```

#include "storage.h"

#include <fstream>
#include <iostream>
#include <sstream>

static std::vector<std::string> split(const std::string& s, char sep) {
    // Стандартна бібліотека C++ такої функції не має — пишемо самі.
    // Пройдіться по символах: сепаратор -> покласти накопичене і почати заново,
    // інакше -> додати символ. У кінці не забудьте покласти останній шматок.
    // "a,b,c" -> {"a","b","c"};  "a,,c" -> {"a","","c"}
}

std::string serialize_chain(const Blockchain& chain) {
    // ostream. Для кожного блоку:
    //   index | timestamp | prev_hash | nonce | hash | транзакції
    // Транзакції через ';', поля транзакції через ','.
    // У кінці рядка — '\n'.
}

bool deserialize_chain(const std::string& text, Blockchain& chain) {
    // 1. istream + getline по рядках. Порожні рядки пропускайте.
    // 2. split(line, '|') має дати РІВНО 6 частин, інакше:
    //    "Помилка формату: очікувалось 6 полів" і false.
    // 3. Шосту частину (якщо не порожня) розбийте по ';', кожна — по ','.
    //    Має вийти рівно 4 поля, інакше "Помилка формату транзакції" і false.
    // 4. Зберіть блок ДРУГИМ конструктором.
    // 5. Порівняйте block.hash із збереженим (частина 5). Не збіглося ->
    //    "Блок N: збережений hash не збігається з вмістом" і false.
    // 6. Жодного блоку -> false. Інакше chain.blocks = зібране, true.
}

bool save_chain(const Blockchain& chain, const std::string& path) {
    // ofstream, перевірити що відкрився, записати serialize_chain.
}

bool load_chain(Blockchain& chain, const std::string& path) {
    // ifstream, перевірити, прочитати весь вміст (buffer << file.rdbuf()),
    // віддати в deserialize_chain.
}

```

src/wallet.cpp — доповнення · тіла написати самому

```

// Угорі файлу додайте #include <openssl/pem.h> і #include <cstdio>.

bool Wallet::save(const std::string& path) const {
    // fopen на запис, PEM_write_PrivateKey(file, key_, nullptr, nullptr, 0,
    // nullptr, nullptr), fclose. Результат 1 = успіх.
}

bool Wallet::load(const std::string& path) {
    // fopen на читання, PEM_read_PrivateKey(file, nullptr, nullptr, nullptr), fclose.
    // ВАЖЛИВО: конструктор уже згенерував ключ. Перш ніж замінити його
}

```

```
// на завантажений, старий треба звільнити — інакше витік.  
}
```

src/main.cpp

```
#include <iostream>  
#include "blockchain.h"  
#include "storage.h"  
#include "wallet.h"  
  
int main() {  
    // 1. Гаманець: якщо файл є — завантажуюмо, інакше створюємо і зберігаємо.  
    Wallet me;  
    if (me.load("wallet.pem")) {  
        std::cout << "Гаманець завантажено з wallet.pem\n";  
    } else {  
        me.save("wallet.pem");  
        std::cout << "Створено новий гаманець, збережено у wallet.pem\n";  
    }  
  
    // 2. Ланцюг: так само.  
    Blockchain chain(3);  
    if (load_chain(chain, "chain.txt")) {  
        std::cout << "Ланцюг завантажено з chain.txt: " << chain.blocks.size() << "  
    блоків\n";  
    } else {  
        std::cout << "Починаємо новий ланцюг\n";  
    }  
  
    // 3. Майнимо ще один блок і зберігаємо.  
    chain.mine_pending(me.address());  
    save_chain(chain, "chain.txt");  
  
    std::cout << "Блоків: " << chain.blocks.size()  
        << ", мій баланс: " << chain.balance_of(me.address())  
        << ", валідний: " << (chain.is_valid() ? "так" : "НІ") << "\n";  
    std::cout << "Запустіть програму ще раз — баланс зросте на 50.\n";  
    return 0;  
}
```

УВАГА

Файл `wallet.pem` — це і є ваші «гроші». У справжній монеті хто має цей файл, той володіє всім на адресі. Наша монета іграшкова, але звичка не комітити `*.pem` у git і не пересилати їх — справжня. Створіть `.gitignore` з рядками `*.pem` і `chain*.txt` просто зараз.

ТВОЄ ЗАВДАННЯ

1. Додайте другий конструктор `Block` і перепишіть перший через делегування.
2. Створіть `storage.h` / `storage.cpp`.
3. Додайте `save` / `load` у `Wallet`.
4. Замініть `main.cpp`, зберіть, запустіть тричі поспіль.

Перевірка

ПЕРЕДБАЧ ДО ЗАПУСКУ

Перед першим запуском: скільки блоків буде після першого запуску? Після другого? Який буде баланс після третього?

ЕТАЛОН ДЛЯ САМОПЕРЕВІРКИ

Три запуски поспіль:

Створено новий гаманець, збережено у `wallet.pem`
Починаємо новий ланцюг
Блоків: 2, мій баланс: 50, валідний: так

Гаманець завантажено з `wallet.pem`
Ланцюг завантажено з `chain.txt`: 2 блоків
Блоків: 3, мій баланс: 100, валідний: так

Гаманець завантажено з `wallet.pem`
Ланцюг завантажено з `chain.txt`: 3 блоків
Блоків: 4, мій баланс: 150, валідний: так

Тепер найголовніше в розділі. Відкрийте `chain.txt` у редакторі, знайдіть у другому рядку `,50,` і замініть на `,5000,` — «намалюйте» собі монет. Запустіть:

Гаманець завантажено з `wallet.pem`
Блок 1: збережений `hash` не збігається з вмістом
Починаємо новий ланцюг
Блоків: 2, мій баланс: 50, валідний: так

Підробку спіймано **на завантаженні**.

Зверніть увагу: програма перезаписала підроблений файл новим ланцюгом. Для іграшки прийнятно, для реальної ноди — ні: там файл лишили б і зупинились із помилкою. Це перша вправа розділу.

САМОПЕРЕВІРКА

1. Навіщо `deserialize_chain` перераховує хеш, якщо він і так є у файлі?
2. Навіщо `Block` знадобився другий конструктор? Чому не вистачило першого?
3. Що робить `EVP_PKEY_free(key_)` у `Wallet::load` і що буде без нього?
4. Чому «текст ↔ ланцюг» і «файл ↔ текст» — різні функції?

Зламай навмисне

ЗЛАМАЙ НАВМИСНЕ

Експеримент 1. Змініть у файлі не суму, а `nonce`. Що скаже завантаження? А якщо змінити `timestamp`? А `prev_hash`?

Експеримент 2. Приберіть перевірку хеша з `deserialize_chain` (крок 5 скелета). Повторіть підробку `,50,` → `,5000,`. Тепер ланцюг завантажується. Чи скаже `is_valid()` про проблему? **Чому ні?** (Підказка: що саме перераховується при завантаженні блоку.)

Експеримент 3. Видаліть `wallet.pem`, залишивши `chain.txt`. Запустіть. Який тепер ваш баланс і куди поділись старі монети?

Експеримент 4. Додайте в кінець `chain.txt` порожній рядок, потім рядок зі сміттям. Обидва випадки обробляються по-різному — знайдіть у скелеті, які саме рядки за це відповідають.

Вправи

1. Замість «почати заново» зробіть так, щоб при невалідному файлі програма виводила помилку і завершувалась із кодом 1.
2. Збережіть у файл ще й чергу `pending`, щоб непідтверджені транзакції не зникали при перезапуску.

Крок 9. Мережа: дві ноди

Мета розділу: програма `node`, яку можна запустити двічі на різних портах; кожна майнить свій ланцюг і вміє забрати ланцюг сусіда, якщо той довший.

Це останній крок, і єдиний, де з'являються потоки та сокети.

Ідея

Нода — це три речі разом:

1. **ланцюг** у пам'яті (і на диску, як у попередньому розділі);
2. **сервер**, який відповідає сусідам на запит «дай ланцюг»;
3. **консоль** із командами `mine`, `send`, `sync`.

Сервер і консоль мають працювати одночасно: поки ви набираєте команду, сусід може попросити ланцюг. Тому сервер живе в окремому **потіці**.

ІДЕЯ

Два потоки торкаються одного ланцюга. Якщо вони зроблять це в ту саму мить — консоль додає блок, а сервер посеред цього серіалізує — сусід отримає половину блоку. Захист — **м'ютекс**: замок, який у кожен момент тримає лише один потік.

Синхронізація. `sync` просить у сусіда ланцюг, розбирає його тією ж `deserialize_chain`, перевіряє `is_valid()` і приймає, лише якщо він **довший** за свій. Це **правило найдовшого ланцюга** у найпростішій формі.

Протокол навмисно примітивний: клієнт надсилає рядок `CHAIN\n` і закриває свою половину з'єднання; сервер відповідає всім текстом ланцюга і закриває з'єднання. Кінець даних = закриття сокета. Жодних довжин, заголовків, JSON.

Нові слова

слово	що означає
<code>std::thread</code>	Потік виконання. <code>std::thread(f, a, b).detach()</code> — запустити <code>f(a,b)</code> у фоні й відпустити.
<code>std::mutex</code>	Замок. Один потік тримає — інші чекають.
<code>std::lock_guard</code>	Бере замок при створенні й автоматично відпускає при виході з області видимості. Забути відпустити неможливо.
<code>socket</code> / <code>bind</code> / <code>listen</code> / <code>accept</code>	POSIX-сокети: створити, зайняти порт, слухати, прийняти клієнта.
<code>connect</code> / <code>send</code> / <code>recv</code>	Клієнтський бік: під'єднатись, надіслати, прочитати.
<code>shutdown(sock, SHUT_WR)</code>	«Я все сказав» — закрити свою половину, щоб сервер побачив кінець запиту.

слово	що означає
sockaddr_in	Структура адреси. <code>INADDR_LOOPBACK</code> = лише 127.0.0.1.
htons / htonl	Порядок байтів мережі. Без них порт зчитується задом наперед.
std::string (*f)()	Вказівник на функцію : сервер не знає про <code>Blockchain</code> , він викликає те, що йому дали.
argc, argv	Аргументи командного рядка: <code>argv[1]</code> — перший після назви програми.

Контракт

src/network.h

```
#pragma once
#include <string>

// Відкриває TCP-порт і чекає на підключення. На запит "CHAIN\n" відповідає
// текстом ланцюга. Працює нескінченно — запускайте в окремому потоці.
// `get_chain` — функція, яка повертає актуальний текст ланцюга.
void run_server(int port, std::string (*get_chain)());

// Підключається до сусіда на localhost:port, надсилає "CHAIN\n"
// і повертає отриманий текст. Порожній рядок = помилка з'єднання.
std::string request_chain(int port);
```

ЧОМУ САМЕ ТАК

`get_chain` — вказівник на функцію, а не посилання на `Blockchain`. Завдяки цьому модуль мережі **взагалі не знає** про існування блокчейну: він викликає те, що йому дали, і надсилає результат. Це називають розв'язанням залежностей, і саме тому `network.cpp` не включає жодного нашого заголовка, крім власного.

Скелет

src/network.cpp · тіла написати самому

```
#include "network.h"

#include <arpa/inet.h>
#include <netinet/in.h>
#include <sys/socket.h>
#include <unistd.h>

#include <cstring>
#include <iostream>

static std::string read_all(int sock, bool stop_at_newline) {
    // Цикл: гесч у буфер на 4096 байтів, дописувати в результат.
    // n <= 0 -> з'єднання закрито, вихід.
    // Якщо stop_at_newline і в результаті вже є '\n' -> теж вихід.
}

static void send_all(int sock, const std::string& data) {
```

```

    // Цикл, поки не відправлено все: send(sock, data.data() + sent,
    // data.size() - sent, 0). n <= 0 -> вихід.
}

void run_server(int port, std::string (*get_chain)()) {
    // 1. socket(AF_INET, SOCK_STREAM, 0)
    // 2. setsockopt(..., SO_REUSEADDR, ...) — щоб перезапуск не чекав хвилину
    // 3. sockaddr_in addr{}; sin_family = AF_INET;
    //    sin_addr.s_addr = htonl(INADDR_LOOPBACK); sin_port = htons(port);
    // 4. bind — не вдалося -> надрукувати помилку і вийти
    // 5. listen(server, 8)
    // 6. Нескінченний цикл: accept -> read_all(client, true) ->
    //    якщо запит починається з "CHAIN" -> send_all(client, get_chain())
    //    -> close(client)
}

std::string request_chain(int port) {
    // Дзеркально: socket, connect (не вдалося -> close і "" ),
    // send_all("CHAIN\n"), shutdown(sock, SHUT_WR),
    // read_all(sock, false), close, повернути прочитане.
}

```

УВАГА

`send` і `recv` **не гарантують**, що передадуть усе за один виклик — лише частину. Саме тому `send_all` і `read_all` — цикли. Це класична пастка сокетів; у власному коді про неї забувають майже завжди.

src/main.cpp — нода · тіла написати самому

```

// Нода: node <мій_порт> [порт_сусіда]

#include <iostream>
#include <mutex>
#include <sstream>
#include <thread>

#include "blockchain.h"
#include "network.h"
#include "storage.h"
#include "wallet.h"

static Blockchain g_chain(3);
static std::mutex g_mutex; // ланцюг читають два потоки: сервер і консоль

static std::string current_chain_text() {
    // Взяти замок через lock_guard і повернути serialize_chain(g_chain).
}

int main(int argc, char** argv) {
    // 1. argc < 2 -> надрукувати підказку і повернути 1.
    // 2. my_port = stoi(argv[1]); peer_port = argc >= 3 ? stoi(argv[2]) : 0.
    // 3. Імена файлів свої для кожної ноди:
    //    "wallet-<порт>.pem" і "chain-<порт>.txt" — інакше дві ноди
    //    в одній папці затруть одна одну.
    // 4. Завантажити гаманець (немає -> зберегти новий) і ланцюг.
    // 5. std::thread(run_server, my_port, current_chain_text).detach();
    // 6. Цикл читання команд зі std::cin. На початку кожної ітерації —
    //    lock_guard на g_mutex.
    //

```

```
// address -> надрукувати me.address()
// balance -> g_chain.balance_of(me.address())
// mine -> mine_pending(me.address()), save_chain, повідомити номер блоку
// send <адреса> <сума> -> зібрати транзакцію, підписати, add_transaction
// chain -> g_chain.print()
// sync -> див. нижче
// quit -> вийти з циклу
//
// sync:
// peer_port == 0 -> "Сусіда не вказано"
// text = request_chain(peer_port)
// Blockchain candidate(g_chain.difficulty);
// порожньо або не розібралось -> "Не вдалося отримати ланцюг сусіда"
// !candidate.is_valid() -> "Ланцюг сусіда невалідний – ігноруємо"
// не довгий за наш -> "У сусіда не довше (X проти Y)"
// інакше -> замінити g_chain.blocks, зберегти, повідомити
}
```

ЧОМУ САМЕ ТАК

`sync` збирає окремий ланцюг `candidate` і підміняє свій лише після всіх перевірок. Свій ланцюг не чіпаємо, поки не впевнились у чужому — інакше невдала синхронізація залишила б ноду з половиною чужих даних.

Скрипт збірки змінюється: виконуваний файл тепер називається `node`.

build.sh

```
#!/bin/sh
set -e
OPENSSL="$(brew --prefix openssl@3)"
clang++ -std=c++17 -Wall -Wextra -pthread \
  -I"$OPENSSL/include" -L"$OPENSSL/lib" -lcrypto \
  -o node src/*.cpp
echo "OK: ./node"
```

ТВОЄ ЗАВДАННЯ

1. Створіть `network.h` / `network.cpp`.
2. Перепишіть `main.cpp` як ноду.
3. Оновіть `build.sh`, зберіть.
4. Відкрийте два термінали в папці проєкту.

Перевірка

ПЕРЕДБАЧ ДО ЗАПУСКУ

Нода А змайнила 2 блоки. Нода Б щойно запустилась. Що станеться, якщо:

1. Б виконає `sync` ?
2. Б виконає `sync` вдруге одразу після першого?
3. А виконає `sync` (сусід не вказано)?

ЕТАЛОН ДЛЯ САМОПЕРЕВІРКИ

Термінал 1 — нода А на порту 5001:

```
./node 5001
Нода на порту 5001, блоків: 1
> mine
Змайнено блок #1
> mine
Змайнено блок #2
> balance
100
```

Термінал 2 — нода Б на порту 5002, сусід 5001:

```
./node 5002 5001
Нода на порту 5002, блоків: 1
> chain
#0 nonce=1004 hash=000766957e86baba...
> sync
Прийнято ланцюг сусіда: 3 блоків
> balance
0
> sync
У сусіда не довше (3 проти 3)
```

Головне, що треба побачити:

- після `sync` у Б з'явився ланцюг А, включно з генезисом А (номери нонсів змінились — це вже інший генезис);
- баланс Б дорівнює нулю: монети змайнив А, і в ланцюгу вони записані на його адресу;
- повторний `sync` нічого не робить — правило найдовшого ланцюга.

САМОПЕРЕВІРКА

1. Навіщо потрібен м'ютекс, якщо команди в консолі виконуються по черзі?
2. Що робить `.detach()` і що було б без нього?
3. Чому `send_all` і `read_all` — цикли, а не один виклик?
4. Чому `sync` перевіряє `is_valid()` сусідського ланцюга, хоча той уже перевірів себе сам?
5. Чому баланс Б після синхронізації нульовий, хоча блоків стало три?

Зламай навмисне

ЗЛАМАЙ НАВМИСНЕ

Експеримент 1. Приберіть `shutdown(sock, SHUT_WR)` з `request_chain`. Запустіть `sync`. Програма зависне — поясніть чому. (Підказка: як сервер розуміє, що запит закінчився?)

Експеримент 2. Змініть `read_all(client, true)` на `read_all(client, false)` у сервері. Той самий симптом, інша причина. Яка?

Експеримент 3. Приберіть `lock_guard` із `current_chain_text`. Запустіть А, і поки він майнить зі складністю 6, викличте `sync` з Б кілька разів. Зловіть бити відповідь.

Експеримент 4. Змайніть 3 блоки на А і 5 на Б, потім `sync` на А. Хто переміг? Тепер змайніть 6 на А і повторіть `sync` з Б. Це і є боротьба ланцюгів у мініатюрі.

Експеримент 5. Приберіть перевірку `candidate.is_valid()` із `sync`. Запустіть ноду, яка віддає навмисно зіпсований ланцюг (відредагуйте її `chain-*.txt` і перезапустіть). Що станеться з чесною нодою?

Що тут спрощено

Щоб ви розуміли межу між іграшкою і справжньою мережею:

1. **Тільки localhost.** `INADDR_LOOPBACK` навмисно забороняє з'єднання ззовні.
2. **Немає розсилки.** Справжня нода сама повідомляє сусідів про новий блок; наша чекає на ручний `sync`.
3. **Немає обміну транзакціями.** Черга `pending` у кожної своя.
4. **Немає списку сусідів.** Рівно один сусід, заданий аргументом.
5. **Немає перевірки роботи при синхронізації складності.** Обидві ноди мають однакову фіксовану складність; у справжній мережі це узгоджується правилами.
6. **Протокол без версій і довжин.** Перше, що зламається при будь-якій зміні формату.

Куди далі

Ви маєте працюючу монету. Дев'ять кроків позаду, і Bitcoin більше не магія, а програма.

Покращити toychain

Від простого до складного:

1. **Автоматична складність.** Підлаштовувати `difficulty` так, щоб блок знаходився раз на N секунд. У Bitcoin — перерахунок кожні 2016 блоків.
2. **Халвінг.** Нагорода зменшується вдвічі кожні N блоків. Порахуйте максимальну емісію.
3. **Комісії.** Майнер отримує нагороду плюс комісії транзакцій блоку. Додайте вибір найвигодніших транзакцій із черги.
4. **Обмін транзакціями по мережі.** Команда `broadcast`, яка розсилає `pending` сусідам.
5. **Список сусідів.** Кілька портів замість одного, `sync` до всіх.
6. **JSON замість власного формату.** Візьміть `nlohmann/json` — одна заголовкова бібліотека, і серіалізація стає надійною.
7. **Модель UTXO замість балансів.** Найбільша зміна: транзакції посилаються на конкретні невитрачені виходи. Це те, як працює Bitcoin насправді.
8. **Тести.** `doctest` або `Catch2` — одна заголовкова бібліотека. Почніть із `merkle_root` і `deserialize_chain`: у них найбільше крайових випадків.

Читати справжній код

- `libsecp256k1` — бібліотека підписів Bitcoin. Невелика, вилізана, з чудовими коментарями.
- **Bitcoin Core**, файл `src/primitives/block.h` — ви впізнаєте кожне поле.
- **Ethereum (geth)** — Go, але модель рахунків та сама, що у вас.

Мова

Якщо C++ зачепив, наступні теми в порядку користі: `std::unique_ptr` і RAII, шаблони, рухома семантика (`&&`), `std::map` / `unordered_map`, алгоритми `<algorithm>`, `std::optional`. `learncpp.com` покриває все це послідовно і безкоштовно.

І одна порада наостанок. Те, що ви щойно зробили — прочитали чужу ідею і відтворили її своїм кодом, ламаючи й передбачаючи, — і є єдиний спосіб вчитись програмуванню, який працює. Наступний проєкт беріть так само.

Додаток А. Типові помилки та як їх читати

Головне вміння — визначити, **на якому з трьох етапів** (§0.2) виникла помилка. Далі все стає простіше.

Ознака	Етап
Названо файл і номер рядка, слово error:	Компіляція
Undefined symbols , ld: , рядка немає	Лінкування
Помилка з'явилась після ./toychain	Виконання

Компіляція

помилка

```
error: use of undeclared identifier 'std'
 1 | int main(){ std::string s = "hi"; }
   |               ^
```

Забутий **#include**. Тут — **<string>**. Компілятор не знає слова, бо йому не показали, де воно описане.

помилка

```
error: use of undeclared identifier 'nonce'
```

Змінна не оголошена, оголошена нижче за місцем використання, або друкарська помилка в назві. Перевірте написання і область видимості.

помилка

```
error: expected ';' after struct
 1 | struct B{ int i; }
   |               ^
   |               ;
```

Забута крапка з комою після закривальної дужки **struct** або **class**. Компілятор навіть показує, куди її вставити.

помилка

```
error: no matching constructor for initialization of 'Block'
note: candidate constructor not viable: requires 3 arguments, but 2 were provided
```

Створюєте об'єкт не тим набором аргументів, який приймає конструктор. Читайте рядок **note:** — там написано, скільки і яких аргументів очікується.

помилка

```
error: cannot assign to non-static data member within const member function 'f'
note: member function 'B::f' is declared const here
```

Метод позначений `const` (§0.14), а ви в ньому щось змінюєте. Або приберіть `const`, або не змінюйте — залежно від того, що правильно.

помилка

```
fatal error: 'openssl/evp.h' file not found
1 | #include <openssl/evp.h>
  | ~~~~~
```

Компілятор не знає, де шукати OpenSSL. Перевірте, що `brew --prefix openssl@3` друкує шлях, і що `build.sh` використовує саме його у `-I`. Найчастіша причина — збірка напряму через `clang++ src/main.cpp` замість `./build.sh`.

Лінкування

помилка

```
Undefined symbols for architecture arm64:
  "sha256(std::__1::basic_string<...> const&)", referenced from:
    _main in main-3a5e6e.o
ld: symbol(s) not found for architecture arm64
```

Функція **оголошена** (компілятор повірив), але тіла ніде немає. Три причини за частотою:

1. Забули передати `.cpp` компілятору. `build.sh` бере `src/*.cpp`, тож перевірте, що файл справді лежить у `src/`.
2. Тіло є, але сигнатура не збігається з оголошенням — наприклад, у заголовку `const std::string&`, а в `.cpp` просто `std::string`. Для лінкера це дві різні функції.
3. Забули `Block::` перед іменем методу в `.cpp` — тоді ви написали вільну функцію, а не метод.

помилка

```
ld: library 'crypto' not found
```

Забутий або неправильний `-L` чи `-lcrypto` у `build.sh`.

Під час виконання

помилка

```
libc++abi: terminating due to uncaught exception of type
std::out_of_range: basic_string
```

`substr` за межами рядка. Найчастіше — `hash.substr(0, difficulty)`, коли `hash` ще порожній. Перевірте, що поле встигли ініціалізувати.

помилка

```
libc++abi: terminating due to uncaught exception of type
std::invalid_argument: stoi: no conversion
```

`std::stoi` / `stol` / `stoll` отримали рядок, який не є числом. У Кроці 8 це означає зіпсований або неповний рядок у `chain.txt`.

помилка

```
zsh: segmentation fault ./toychain
```

Звернення за неправильною адресою. У цій книзі майже завжди — робота з уже звільненим або нульовим вказівником OpenSSL: використали `ctx` після `EVP_MD_CTX_free`, або не перевірили результат на `nullptr`.

УВАГА

Помилка, якої немає. Вихід за межі вектора не викликає жодного повідомлення:

```
std::vector<int> v{1, 2, 3};  
std::cout << v[10];    // друкує 0, програма продовжує працювати
```

Це **undefined behavior**: програма може надрукувати сміття, впасти пізніше в зовсім іншому місці, або працювати «правильно» роками. Якщо поведінка незрозуміла і ніщо не падає — шукайте вихід за межі.

Додаток Б. Шпаргалка

Файли проєкту (фінальний стан)

структура

```
toychain/
  build.sh
  .gitignore
  UNKNOWN.md
  src/
    sha256.h      sha256.cpp      Крок 1
    block.h       block.cpp      Кроки 2, 4, 5, 8
    blockchain.h  blockchain.cpp  Кроки 3, 4, 5, 7
    transaction.h transaction.cpp  Кроки 5, 6
    merkle.h      merkle.cpp      Крок 5
    wallet.h      wallet.cpp      Кроки 6, 8
    storage.h     storage.cpp      Крок 8
    network.h     network.cpp      Крок 9
    main.cpp      змінюється кожного кроку
```

П'ять перевірок `is_valid()`

Ланцюг валідний, якщо для **кожного** блоку, починаючи з другого, виконано все:

1. `hash == calculate_hash()` — вміст не змінювали (**Крок 3**);
2. `prev_hash == попередній.hash` — блоки зчеплені (**Крок 3**);
3. `hash` починається з `difficulty` нулів — роботу виконано (**Крок 4**);
4. `merkle_root == merkle_root(transactions)` — транзакції не підмінили (**Крок 5**);
5. кожна транзакція має вірний підпис (**Крок 7**).

Кожна перевірка закриває дірку, яку відкривала попередня версія. Приберіть будь-яку — і знайдеться атака, яка через неї пройде. Це вправа «Зламай» кожного з цих кроків.

Команди

```
./build.sh      зібрати
./toychain      запустити (кроки 1-8)
./node 5001     нода без сусіда (крок 9)
./node 5002 5001 нода з сусідом
echo -n hello | shasum -a 256  незалежна перевірка SHA-256
brew --prefix openssl@3      шлях до OpenSSL
```

Команди ноди

```
address      моя адреса (130 hex-символів)
balance      мій баланс
mine         змайнити блок з черги, нагорода мені
```

```
send <адреса> <сума>
chain                показати ланцюг
sync                забрати ланцюг сусіда, якщо він довший і валідний
quit
```

Додаток В. Словник понять C++

Чекліст. Ставте позначку, лише коли можете пояснити поняття вголос **своїми словами**, без підглядання. Порожня клітинка навпроти пункту — це план на наступний сеанс, а не привід засмучуватись.

✓	§	Поняття	Питання, на яке треба вміти відповісти
☐	0.2	Три етапи збірки	Чим помилка лінкера відрізняється від помилки компілятора?
☐	0.4	Типи	Чому гроші — <code>long long</code> , а не <code>double</code> ?
☐	0.5	<code>const</code> (значення)	Що перевіряє компілятор?
☐	0.6	<code>std::string</code>	Що робить <code>substr(0, 4)</code> ?
☐	0.7	Функції	Чим оголошення відрізняється від визначення?
☐	0.8	<code>& i const&</code>	Чому <code>const&</code> швидший за копію?
☐	0.9	Цикли	Що означає <code>for (const Block& b : blocks)</code> ?
☐	0.10	<code>std::vector</code>	Що поверне <code>v[10]</code> у векторі з трьох елементів?
☐	0.11	<code>struct</code>	Навіщо крапка з комою в кінці?
☐	0.12	<code>class</code> , <code>private</code>	Єдина різниця між <code>struct</code> і <code>class</code> ?
☐	0.13	Список ініціалізації	Чому <code>index(index)</code> — не нескінченний цикл?
☐	0.14	<code>const</code> -метод	Чим відрізняється від <code>const</code> у параметрі?
☐	0.15	<code>.h</code> / <code>.cpp</code>	Що робить <code>#pragma once</code> ?
☐	0.16	Вказівники	Навіщо <code>EVP_MD_CTX_free</code> ?
☐	0.17	Потоки	Який маніпулятор «нелипкий» і чому це важливо?
☐	0.18	Приведення типів	Чому <code>(int)</code> перед байтом обов'язкове?
☐	0.19	<code>static</code> , <code>throw</code>	Що означає <code>static</code> перед функцією в <code>.cpp</code> ?

Додаток Г. Глосарій

Хеш	Відбиток даних фіксованої довжини. SHA-256 дає 32 байти = 64 hex-символи.
Лавинний ефект	Зміна одного біта на вході змінює приблизно половину бітів хеша.
Блок	Структура з даними, часом, посиланням на попередника і власним хешем.
Генезис	Перший блок ланцюга. Не має попередника, тому <code>prev_hash = "0"</code> .
Ланцюг	Послідовність блоків, де кожен містить хеш попереднього.
Nonce	Число, яке майнер підбирає, щоб хеш блоку почав відповідати умові.
Proof of Work	Доказ виконаної роботи. Знайти — дорого, перевірити — дешево.
Складність	Скільки нулів має бути на початку хеша. Кожен нуль — у 16 разів дорожче.
Майнінг	Підбір nonce до виконання умови складності.
Транзакція	Запис «від кого, кому, скільки», підписаний відправником.
Mempool	Черга транзакцій, які ще не потрапили в блок. У нас — <code>pending</code> .
Coinbase	Транзакція без відправника — нагорода майнеру. Єдине джерело нових монет.
Дерево Меркла	Попарне хешування транзакцій до одного кореня.
Корінь Меркла	Один рядок, що представляє всі транзакції блоку в його хеші.
Приватний ключ	Випадкове 256-бітне число. Хто його має — володіє монетами.
Публічний ключ	Точка на кривій, обчислена з приватного. Зворотно — неможливо.
Адреса	У нашій монеті — публічний ключ у hex (130 символів).
secp256k1	Еліптична крива, яку використовують Bitcoin та Ethereum.
ECDSA	Алгоритм цифрового підпису на еліптичних кривих.
Підпис	Рядок, що доводить: власник ключа схвалив саме це повідомлення.
Подвійна витрата	Спроба витратити ті самі монети двічі. Ловиться врахуванням <code>pending</code> .

Модель рахунків	Баланс = сума входів мінус виходів. Так робить Ethereum і наша монета.
UTXO	Модель Bitcoin: не баланси, а невитрачені виходи попередніх транзакцій.
Халвінг	Зменшення нагороди вдвічі кожні N блоків. Обмежує загальну емісію.
Серіалізація	Перетворення об'єктів у текст (або байти) і назад.
РЕМ	Текстовий формат зберігання ключів, який читає й пише OpenSSL.
Нода	Учасник мережі: ланцюг + сервер + консоль.
Правило найдовшого ланцюга	За правду вважається найдовший валідний ланцюг.
М'ютекс	Замок, який у кожен момент тримає лише один потік.